

ТД Управления версиями

История изменений технической документации

Что было изменено	Кто изменил	Когда
Добавлена информация по анализу ревизий модулей и работе с версиями модулей при обновлении сервиса в раздел "Утилиты обновления"	Кузнецов А.П.	24.10.2019
Актуализация документа по результатам реализации следующих проектов: Последовательное обновление узлов сервиса , Обновление сервисов с равноправными базами и Автоматизация развертывания web - приложений по описанию из дистрибутива	Евстифеев А.С.	26.12.2019
Актуализация документа по результатам реализации проекта Обновление сервисов с равноправными базами	Евстифеев А.С.	08.04.2020
Актуализация документа по результатам реализации в Хоттабыче проекта Вынос деплоя jinnee_lite на сборочные серверы	Евстифеев А.С.	16.10.2020
Актуализация документа по результатам реализации в Хоттабыче проекта Вынос деплоя jinnee_lite на сборочные серверы	Кузнецов А.П.	31.10.2020
Актуализация документа по результатам реализации в Хоттабыче проекта Повышение удобства посхемных обновлений	Дмитерко С.И.	12.03.2021
Актуализация документа по результатам реализации в Хоттабыче проекта Контроль нагрузки на серверы конвертации	Дмитерко С.И.	16.06.2021
Актуализация документа по результатам реализации в Хоттабыче проекта Динамические группы равноправных баз на фазе обновления	Евстифеев А.С.	17.06.2021
Актуализация документа по результатам реализации в Хоттабыче проекта Декомпозиция управления версиями и увеличение асинхронности	Дмитерко С.И.	15.01.2024
Добавлена схема первичного разворота сервиса на узлах ВМ	Кузнецов А.П.	31.05.2024
Добавлен раздел "Подготовка конвертации БД" по проекту Проведение подготовки конвертации	Кузнецов А.П.	10.06.2024

основных сервисов на нефинальном дистрибутиве		
Реорганизована документация по системе обновлений.	Кузнецов А.П.	27.02.2025
Исправление замечаний по поручению .	Дмитерко С.И.	27.05.2025
Добавлен раздел по проекту Обновление web-сервисов при неработающем RabbitMQ .	Кузнецов А.П.	24.09.2025
Добавлен раздел по поручению .	Сиряков М.К.	12.01.2026

Краткое описание продукта/подсистемы

Основная цель — систематизация процессов обновлений продуктов компании.

Подавляющее большинство приложений написано на платформе СБИС и описано в структуре управления облаком. Логическое и физическое описание облачных приложений позволяет систематизировать процесс их обновления и автоматизировать процесс развёртывания по описанию из дистрибутива.

Большое число облачных приложений, различия в их реализации и сценариях использования привело к необходимости реализации различных типов обновления, а функционал по исполнению скриптов, даёт возможность разработчикам сервисов на платформе СБИС выполнять произвольный SQL и Python код на рабочих серверах, контролируя этот процесс.

Для приложений, которые в облаке описаны быть не могут (например, находятся за пределами нашей сети, десктопные приложения), но должны входить в релизный цикл компании и обновляться централизованной системой, реализован сервис update.sbis.ru, который предоставляет возможность распределенного скачивания файлов по заранее известным ссылкам. Задачей Хоттабыча служит систематизация хранения данных на update.sbis.ru, их доставка на мастер-сервер при запуске обновления, запуск и контроль за синхронизацией мастера по зеркалам.

Термины и определения

Задача — отдельное законченное действие, которое необходимо выполнить в ходе обновления. К примеру: составить граф задач для фазы, загрузить файлы на обновляемые серверы, запустить утилиту обновления с заданными параметрами, запустить выполнение скрипта, зарегистрировать задачу планировщика и т.д.

Менеджер асинхронного обновления (MAO) – функционал по управлению графом задач (функции определены в файле `async_update_manager.cpp`: создание графа задач, отбор и исполнение задач и т.д.), по сути с его помощью происходит управление обновлением.

Облачное приложение – приложение, написанное на платформе СБИС или на других фреймворках и описанное в структуре управления облаком.

Тип обновления — алгоритм обновления приложения. Разбиваются на две группы: легкие и блокирующие.

Логика — отдельная логическая единица, описывающая структуру каждого сервиса в отдельности и приложения в целом. Исходя из типа логики определяются действия, которые необходимо выполнить на каждой фазе процесса обновления.

Хоттабыч – неофициальное название сервиса "Управление версиями".

Модуль выполнения скрипта — модуль, содержащий ORX с методом `DeveloperScript.ExecuteAll`. Данный метод запускает непосредственно выполнение скрипта разработчика.

update.sbis.ru – географически распределенный сервис, предоставляющий загрузку клиентскими приложениями данных с сайта по заранее определенным ссылкам.

Балансировщик – сервис на Node JS, контролирующий работу мастер-сервера и зеркал. Управляет распределением нагрузки по зеркалам, синхронизацией между ними.

Мастер-сервер – основной сервер, данные которого считаются эталоном для синхронизации по зеркалам. Именно на нем Хоттабыч размещает новые файлы.

Зеркало – один из дополнительных серверов в системе update.sbis.ru, который после окончания репликации содержит те же данные, что и мастер. Зеркало забирает часть нагрузки с мастер сервера по раздаче контента. Может географически находиться ближе к получателю, тем самым сокращая время получения данных.

Пакет данных – zip-архив, содержащий некоторые данные.

Ревизия модуля – некоторый хэш модуля, вычисляемый джином при сборке дистрибутива, на основе структуры файлов и каталогов модуля и данных в файлах. Влияет на то, нужно ли дублировать физическое хранение модуля в хранилище, либо перезаливать файл на обновляемый сервер.

Версия модуля – характеристика, указывающая на минимальную версию дистрибутива, в котором в последний раз вносились правки по модулю. Влияет на то, будет ли перекачивать клиент файлы модуля, либо будет брать их из кэша.

Хранилище дистрибутивов (ХД) — в рамках данного документа имеется в виду сервис distribution storage продукта "Хранилище дистрибутивов", который служит для организации и хранения файлов дистрибутивов сервисов. Главная цель сервиса – хранение артефактов сборки и предоставление файлов дистрибутивов сервису "Управление версиями" для обновления других сервисов.

Патч — zip-архив, создаваемый разработчиком для модификации версии дистрибутива. В контексте приложений СБИС, дистрибутивом называют именованное файловое представление сервиса (программного продукта), которое подлежит установке. А сама установка выполняется через Хоттабыч.

Концептуальное описание решения и основные алгоритмы

Обновление облачных приложений

Как следует из определения облачного приложения, его структура описана в облаке. Исходя из этого описания и заданных пользователем настроек обновления доступны те или иные типы обновления. Тип обновления выбирается при запуске обновления и именно от него зависит, что именно и в какой последовательности будет выполнено при обновлении приложения.

Скрипты

Разработчик оформляет код в виде модуля, содержащего бизнес-объект DeveloperScript с методами в формате ORX – [справочника объектов](#), содержащего описание объектов и [методов](#), при помощи которых можно организовать бизнес-логику веб-приложения. Модуль разработчика внедряется на бизнес-логику сервиса. Затем Хоттабыч вызывает служебный метод на сервисе, который вызывает все методы скриптов. Вызовы делаются для каждого клиента или сервиса с помощью [сервиса DWC](#).

Необлачные приложения

В облаке присутствует балансировщик, админка которого прописана в структуре. В Хоттабыче задается связь приложения балансировщика и облака. В Хоттабыче заводятся необлачные приложения, запуск которых будет инициировать загрузку данных на update.sbis.ru.

Процесс загрузки определяет мастер-сервер балансировщика и через агента обновления, установленного на нем, доставляет заливаемый дистрибутив. На фазе обновления утилита агента подменяет данные в папке мастер-сервера и инициирует через API-балансировщика процедуру синхронизации зеркал. Агент дожидается окончания синхронизации, что сигнализирует об окончании фазы обновления. На подтверждении агент удаляет бэкапы выгрузки и остальные временные файлы.

Управление обновлением

Управление обновлением обрабатывает полученные сигналы от пользовательского интерфейса и, в зависимости от текущего состояния и заданных настроек, формирует и запускает на выполнение задачи или обновляет их состояние.

Развёртывание web-приложения по дистрибутиву

Напрямую из Хоттабыча нельзя развернуть приложения. Данный процесс инициируется в облаке, где происходит анализ состава дистрибутива и структуры облака, после чего производятся действия для реализации создания нового приложения, сервиса, новой бизнес-логики, новой tenant-группы и после производится разворот окружения на указанных серверах. После управление передаётся Хоттабычу, который производит установку сервисов и конвертация баз данных.

Архитектура продукта

Информационная модель

Не актуально.

Сервисы и базовые сущности

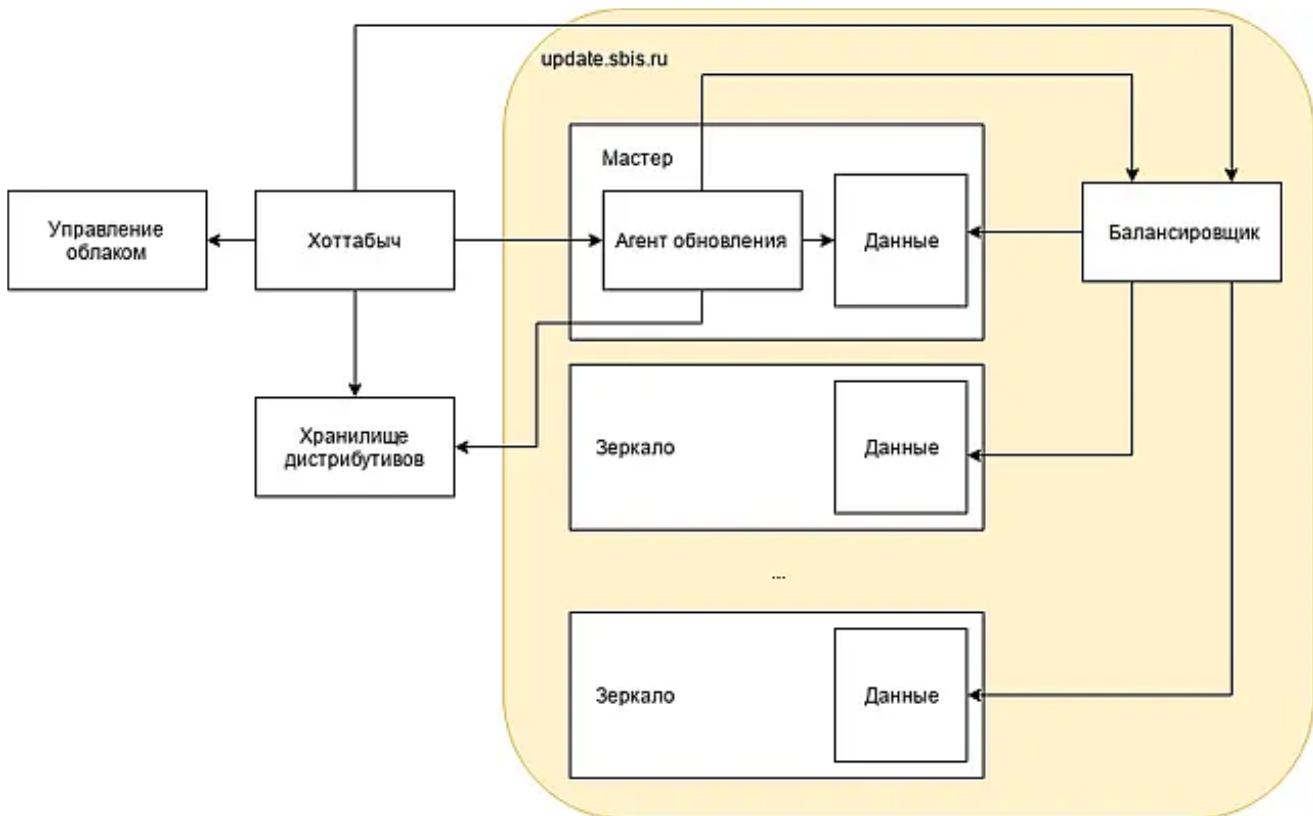


Рисунок 23. Архитектура процессов обновления

Пояснение к схеме архитектуры.

1. Хоттабыч из управления облаком получает информацию для доступа к балансировщику.
2. Хоттабыч загружает и проверяет наличие дистрибутива в ХД.
3. Хоттабыч дает команду агенту обновления для загрузки данных дистрибутива из ХД на мастер-сервер.
4. Хоттабыч дает команду агенту на запуск синхронизации. Агент при этом дает команду балансировщику, который разливает данные на зеркала. Агент контролирует завершение синхронизации через API балансировщика.

Техническое описание

Управление обновлением

Запуск обновлением происходит через интерфейс окна мастер обновления (рис. 2, 3 и 5), посредством нажатия кнопок, что приводит к RPC-вызову функций:

- DeployPackage.Start — для развёртывания данных,
- Продукт.Install — для первичного развёртывания;
- Продукт.Обновить для остальных типов обновления.

Внутри данных функций выполняется первичная проверка и подготовка для выполнения заданных пользователем действий, после чего управление передаётся менеджеру асинхронного обновления(МАО) для формирования первичного графа задач. МАО для заданного действия формирует в БД граф задач, состоящий только из задач по формированию графа для фазы(PhaseFormationTask), результатом выполнения каждой из которых, является сформированный в БД граф задач для заданной фазы.

Управление обновлением реализовано с помощью ориентированного ациклического графа, где вершины графа — это задачи (табл. "Task"), а дуги — зависимости между ними (табл. "TaskRelationship"). Построение графа для каждой фазы происходит при запуске задачи PhaseFormationTask, которая добавляется в граф либо в ходе запуска процесса обновления, либо при запуске следующей фазы. Для построения графа используется библиотека boost graph library (bgl), а именно, класс adjacency_list, шаблонные параметры которого заданы таким образом, что граф является ориентированным, дуги сохраняются в std::list, набор вершин - в std::vector. Так же заданы следующие свойства для вершин: указатель на задачу и цвет. Первое свойство позволяет по вершине получить задачу, а второе - необходимо для алгоритма обхода в глубину(DFS), который используется для сохранения графа в БД, после того как он будет полностью построен.

После того, как граф сохранён в БД, определяются задачи, которые имеют статус ReportStatus::rsPENDING = 4 и не имеют незавершённых задач, от которых они зависят. Выполняемые задачи помечаются статусами:

- ReportStatus::rsQUEUE = 6 - после того, как задача была отобрана для выполнения,
- ReportStatus::rsRUNNING = 2 - когда идёт непосредственное исполнение задачи

и не могут быть повторно взяты на исполнение, пока пометка не будет снята.

Выполненные задачи помечаются статусом ReportStatus::rsFINISHED = 0, что приводит к разрешению зависимостей в графе и запуску последующих задач.

Хранение графов в БД позволяет продолжить асинхронное управление на любом равноправном узле при выходе из строя одного из узлов БЛ Хоттабыча.

МАО также обрабатывает отчёты агента, обновляя статусы задач для которых пришли отчёты, и, при необходимости, статусы сервисов и обновления. В случае, если отчётов по задачам не поступало в течение заданного времени, то обновление, как и все выполняемые задачи, завершаются ошибкой с соответствующим сообщением.

Основные действия производимые в процессе обновления, разбиты на фазы:

- Доставка – загрузка утилиты обновления на сервера.
- Обновление агентов – обновление агентов на заданную версию.
- Подготовка – производятся подготовительные операции.
- Обновление – непосредственное обновление сервиса.
- Откат – служит для отмены изменений, произведенных Хоттабычем на фазе обновления. Для бизнес-логик, статик происходит восстановление файлов из бэкапа. Откат конвертации баз данных не производится, ввиду технической невозможности восстановления данных в процессе конвертации.
- Очистка – удаление временных файлов и бэкапов. После запуска этой фазы ни на какую другую фазу перейти уже невозможно. В процессе работы фазы агентам логик рассылаются задания на удаление файлов, которые были созданы Хоттабычем в процессе обновления.
- Очистка папок – заключительная фаза обновления продукта. В процессе работы фазы агентам логик рассылаются задания на удаление временных папок.

Остальные действия не разбиваются на фазы:

- Возобновить – возобновить прерванный этап. (заново повторяет выполнение прерванной фазы, исключая логики, которые находятся в завершённом состоянии).
- Прервать – прерывает выполнение текущей фазы. Помечает задачи как “Прервано” и, при необходимости, отправляет задание агенту о прерывание текущего задания, в этом случае статус обновления установится как “Прервано” только после получения ответов от всех агентов, или по истечению таймаута – 30 сек.
- Возобновить ошибочные – повторить задачи, выполнение которых завершилось ошибкой, в случае если имеются другие задачи, выполнение которых продолжается.

Всего существует 11 типов задач (в скобках приведено название класса задачи в исходном коде, и, там где это необходимо, сокращение используемое на схемах):

- формирование графа задач для заданного этапа/фазы (PhaseFormationTask; PFT);
- подфазы. Информировать пользователя о ходе обновления, и выставляют режимы обновления (SubPhaseTask; SPT);
- загрузка утилиты на обновляемый сервер (DownloadUtilityTask);
- запуск утилиты с заданными параметрами (RunUtilityTask; RUT);
- отправка задания контейнеру (ContainerTask);
- обновление агента (UpgradeAgentTask);
- запуск скрипта (ScriptTask);
- очистка кэша (BICacheTask);
- регистрация задач планировщика (SchedulerTask; ST);
- регистрация параметров кластера в облаке (ClusterTask);
- регистрация значений параметров (RegParamValuesTask, RPVT);
- завершение фазы (задачи типа ...FinalizerTask; FT).

Задачи загрузки и запуска утилиты отправляют задания агенту. Для этого используется класс AgentRequest, а именно, его методы DownloadUtility и Async, соответственно. Агент, в процессе выполнения полученных заданий, раз в 5 секунд отправляет отчёты Хоттабычу (RPC-вызов метода Продукт.ОтчетыАгента), которые содержат информацию о ходе их выполнения. Хоттабыч же обрабатывает полученные отчёты и выставляет актуальное состояние логик и задач, а так же обновляет прогресс бар.

Задача обновление агента, используя метод Detached класса AgentRequest, отправляет запрос агенту на самообновление, после чего запрашивает версию агента. И завершает своё выполнение если в ответе указана нужная версия агента.

Задача отправки задания контейнеру, асинхронно вызывает метод `Container.AsyncAction` (из API сервиса `datacenter`) с заданным действием и параметрами, которые формируются в зависимости от обновляемой логики и фазы обновления. Результат выполнения получает в виде отчётов (`Продукт.ОтчетыАгента`).

Задача запуска скрипта производит RPC-вызов метода `UpdateScript.Execute`. А результат выполнения получает в виде отчётов (`Продукт.ОтчетыАгента`), которые формируются по вызову планировщиком метода `ClientDatabase.ProcessScriptResponses`.

Задача сброса кэша производит RPC-вызов метода облака `Nginx.ResetCache` и ожидает завершения его выполнения.

Задачи регистрации задач планировщика и регистрация параметров кластера в облаке производят RPC-вызов заданного параметрами задачи метода и дожидаются его выполнения.

Задач регистрации значений параметров (значения параметров задаются для приложения и его сервисов в дистрибутиве и/или при запуске обновления) производит синхронный RPC-вызов метода облака `Settings.RegisterParameter` для каждой группы (`strict` и `flexible`) заданных параметров.

После того, как пользователь выбрал облако и продукт, для запуска обновления необходимо нажать кнопку "Обновить", что приводит к вызову метода "Продукт.Обновить" с заданными параметрами обновления. Внутри "Продукт.Обновить" происходит вызов метода подготавливающего параметра обновления в процессе выполнения которого:

- проверяется доступность облака для заданного продукта;
- проверяются права пользователя;
- проверяются зависимости. Происходит анализ файла `dependencies.xml` на наличие зависимостей для сервисов выбранных для обновления. У облака, по названиям найденных зависимых сервисов и версии на которую обновляется основной продукт, запрашиваются версии для каждого зависимого сервиса. Контролируется наличие необходимых версионных контекстов;
- проверяется наличие обновляемых логик и дублирующих серверов БЛ и статик;
- проверяется допустимость запуска приложения с данным типом обновления и, если задан тип обновления мастер-реплика, наличие реплик у всех БД приложения;
- проверяется возможность запуска обновления или выполнения скрипта, в зависимости от политики выполнения скриптов и уже запущенных обновлений;
- проверяется необходимость обновления агентов;
- задаются параметры для логик.

После чего управление передаётся МАО, который добавляет в граф задач первые вершины – задачу формирования фаз подготовки, и, в случае непрерывного обновления, фаз обновления и подтверждения. И запускается выполнение графа задач.

В процессе выполнения задачи формирования фаз, в граф добавляются фазы для заданного этапа. Каждая последующая фаза ставится в зависимость от предыдущей, т.е. следующая фаза начнёт выполняться только после завершения предыдущей. В результате выполнения фаз в граф добавляются все задачи, которые необходимо выполнить для завершения данной фазы. Сформированные фазой задачи делятся на подфазы, каждая подфаза заканчивается вспомогательной задачей, которая зависит от всех задач внутри подфазы и служит для информирования пользователя о ходе процесса обновления и сигнализирует о том, что могут быть запущены задачи следующей подфазы. Задачи внутри подфазы могут выполняться как параллельно, так и последовательно (в зависимости от типа обновления и обновляемых логик).

Первый этап в процессе обновления – подготовка. Он разбит на три фазы: доставка, обновление агентов, подготовка.

В процессе выполнения фазы "Доставка" на БЛ Хоттабыча происходит загрузка архива `*_db.zip` из ХД для дистрибутива обновляемого продукта новой версии и его распаковка. Из архива извлекается вся необходимая информация (параметры приложения и его сервисов, данные для регистрации настроек и семейств, задачи планировщика и т.д.) и сохраняется в БД Хоттабыча, после чего формируются задачи по загрузке утилиты обновления (`sbis-update-exec_nix.x86_64.zip` берётся из ХД по версии Хоттабыча, дистрибутив admin.sbis.ru) на каждый из серверов, для которых задан агент.

На фазе "Обновление агентов", происходит проверка актуальности агентов на серверах и, в случае если установленные агенты имеют не актуальную версию, формируются задачи для обновления агентов.

На фазе "Подготовка" происходит построение графа задач для необходимых подготовительных операций:

- для логики "Регистрация задач планировщика" – задача по сохранению deregистрируемых задач планировщика;
- для логики "Регистрация кластеров" – задача по регистрации используемых кластеров;
- для БД и логики "Регистрация параметров" – задача по загрузке на конвертер и распаковки `*_db.zip`, `jinnelite`. Для типов обновления с конвертацией для БД добавляется задача на запуск утилиты, которая запускает подготовку конвертации БД;
- для БЛ — задача для утилиты для загрузки на обновляемый сервер всех изменённых между версиями частей сервиса (ЧС) (или всех частей сервиса, в случае если при запуске был выставлен флаг "Принудительная полная замена файлов" или предыдущее обновление завершилось ошибкой) и их распаковки.

Второй этап в процессе обновления – непосредственно, обновление. Содержит одноимённую фазу, при выполнении которой происходит построение графа задач для непосредственного обновления продукта:

- для БЛ — задача запуска утилиты, в ходе которой происходит: определение, какие ЧС устарели или были удалены между версиями и их перемещение из каталога сервиса во временный каталог; перемещение файлов, полученных в ходе подготовки в каталог сервиса; проверка целостности сервиса (сверка хэша самого сервиса и всех его частей на обновляемом сервере с хэшами из ХД); перезапуск сервиса, его прогрев и проверка версии.
- для БД - конвертация или развёртывание данных.
- для логики "Регистрация параметров" - задача запуска утилиты, в ходе которой происходит запуск jinnеe для регистрации параметров, и задача регистрации значений параметров;
- для логики "Регистрация задач планировщика" - задача по регистрации задач планировщика.

Третий этап – очистка. Данный этап разбит на две фазы: очистка и очистка папок. На этих фазах происходит построение графа задач по удалению временных файлов и папок соответственно.

Отказоустойчивость управления обновлением при отказе RabbitMQ

Система обновлений – технически сложное решение, для удобства управления которым используются различные middleware-сервисы. Среди них есть как критически важные (например, сервис управления облаком или хранилище файлов), так и второстепенные, к которым в частности относятся кластера асинхронного обмена и событий RabbitMQ. Аварии с RabbitMQ могли приводить к парализации работы системы обновлений, а невозможность выкатить хотфикс, патч или применить скрипт может сильно усложнить ликвидацию таких аварий.

Решение позволяет в ограниченном режиме использовать Хоттабыча для проведения обновлений и скриптов. Ограниченность видна сразу в интерфейсе, так как неработающие клиентские события не позволяют интерфейсу автоматически перерисовывать состояние реестров при изменении состояния на БЛ. Но в этом ничего страшного нет, так как в аварийном режиме можно перерисовать интерфейс обновив страницу или переоткрыв панель для того, чтобы данные перечитались.

Сложнее дело обстоит на БЛ. Для минимально необходимого функционирования БЛ при неработающем RabbitMQ нужно задать в облаке следующие значения параметров:

- **Ядро.Асинхронные сообщения.ТранспортныйПротоколПоУмолчанию = HTTP** (для сервисов version manager, datacenter, metadata-registrator, cloud control api).
- **Управление версиями.Обновления.Аварийный режим работы = ДА** (для сервиса version manager)

Первый параметр отвечает за переключение в ядре платформы всех асинхронных вызовов на синхронные без гарантии доставки. Тем самым, все асинхронные вызовы, которые могут быть вызваны внутри этих сервисов, не отправляются в RabbitMQ, а вызываются напрямую. В этом случае могли возникать только проблемы с асинхронными вызовами, на которые навешан колбэк обработки результата вызова. В зависимости от ситуации, такие проблемы были решены.

Второй параметр позволяет сервису управления версиями понимать, что он находится в аварийном режиме, чтобы использовать обходные пути выполнения некоторых действий:

1. Переход на локальный планировщик. При запуске сервиса version manager в аварийном режиме происходит активация работы модуля монитора фоновых операций. Через этот модуль происходит периодический вызов метода, связанного с задачей планировщика, отвечающей за обход графа задач обновлений. Это критически важная задача, без которой ход управления обновлением невозможен. При этом вызов этой же задачи из планировщика глушится, чтобы в случае включения в работу облачного планировщика выполнение задач не дублировалось.
2. Активация самодостаточности работы неперсональных скриптов. Запуск скриптов работает через сценарии DWC, так как они хорошо справляются с задачей обхода клиентов, что и требуется скриптам. Но это для персональных приложений. Неперсональные приложения для общности тоже работают через функционал DWC. Включение параметра аварийной работы исключает dws из цепочки запуска неперсональных скриптов. В этом случае вызов скриптовых методов на скриптовой БЛ и обратные вызовы с публикацией отчетов о ходе выполнения скрипта вызываются напрямую.

Облачные приложения

При запуске обновления его тип записывается в БД ("ИсторияОперацийПродукта.ТипОбновления"). Исходя из значения данного параметра, в функции ForEachLogic (которая определена в abstract_logic_handler.cpp) происходит формирование логики для данного обновления. Список логик сервисов приложения получается вызовом метода облака ServiceNode.ListForUpdate (или ServiceNode.ListForUpdateWithDisabled при запуске развёртывания сервиса, чтобы учесть БД, выведенные на обслуживание) для каждого сервиса. Для формирования логики регистрации параметров в облаке вызывается тот же метод ServiceNode.ListForUpdate, в качестве параметра которому передаётся идентификатор сервиса admin-api, полученного в результате вызова метода облака CloudService.GetServiceIdBySystemName. Так же происходит формирование логики регистрации в облаке используемых продуктом кластеров и логики регистрации задач планировщика (алгоритм описан в методе PrepareHandler::OnLogic). После того, как логики сформированы и выполнены предварительные проверки, запускается процесс формирования графа задач, работа которого подробно описана в документе ТД "Управление обновлением".

Для каждого приложения, дистрибутив которого собран под сервис статистики и имеются интерфейсные модули, добавляются логики серверов соответствующего сервиса статистики. На эти серверы в момент обновления происходит загрузка новых модулей, алгоритм которой описан в [ТД сервиса статистики](#).

Пояснение к схемам графов (рис. 11 — 17):

- для запуска задачи "А" необходимо, чтобы все вершины из которых исходят дуги, входящие в вершину обозначающую задачу "А", имели статус rsFINISHED = 0, а сама задача "А" имела статус rsPENDING = 4 (рис. 10);

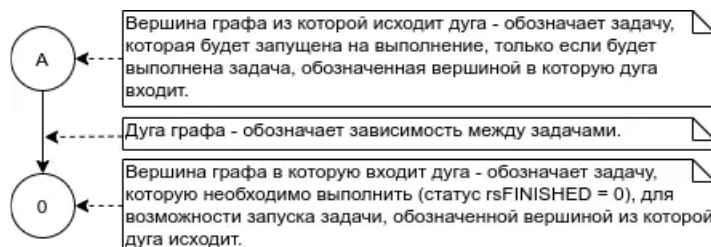


Рисунок 10. Пояснение к схемам графов.

- сокращения, используемые для обозначения вершин графа — есть аббревиатуры задач, которые описаны в разделе "Управление обновлением" настоящего документа.

Легкое обновление

При запуске лёгкого обновления на мастере обновления есть возможность выбора, какие сервисы приложения будут участвовать в обновлении.

При данном обновлении не меняется структура базы данных, а лишь запускается разворот данных Jinnee, поэтому пользователь продолжает работать на протяжении всего процесса обновления. Настройки разворота формируются в методе Preparation::GenerateS3Reg и сохраняются в файле с расширением .s3reg.

Объектами обновления являются серверы статистики и бизнес-логики. В процессе обновления облаку будут отправлены команды по установке режима обновления (mode) umLite = 1 с различными значениями параметра node_mode (метод CloudGroup.SetUpdateMode), что позволяет перенаправить запросы пользователя только к тем серверам, которые в данный момент не обновляются. Перед обновлением первой части серверов БЛ устанавливается режим с параметром node_mode = update_first (рис. 11PFT), который перенаправляет пользователей приложения на вторую часть серверов БЛ. После обновления первой части серверов БЛ устанавливается режим с параметром node_mode = update_both который позволяет пользователям заходить и на те, и на другие сервера, в зависимости от версии (рис. 11, SPT 4). После чего устанавливается режим с параметром node_mode = update_second (рис. 11, SPT 4) и начинается обновление второй части. По окончании основной фазы, режим обновления снимается (рис. 11, FT).

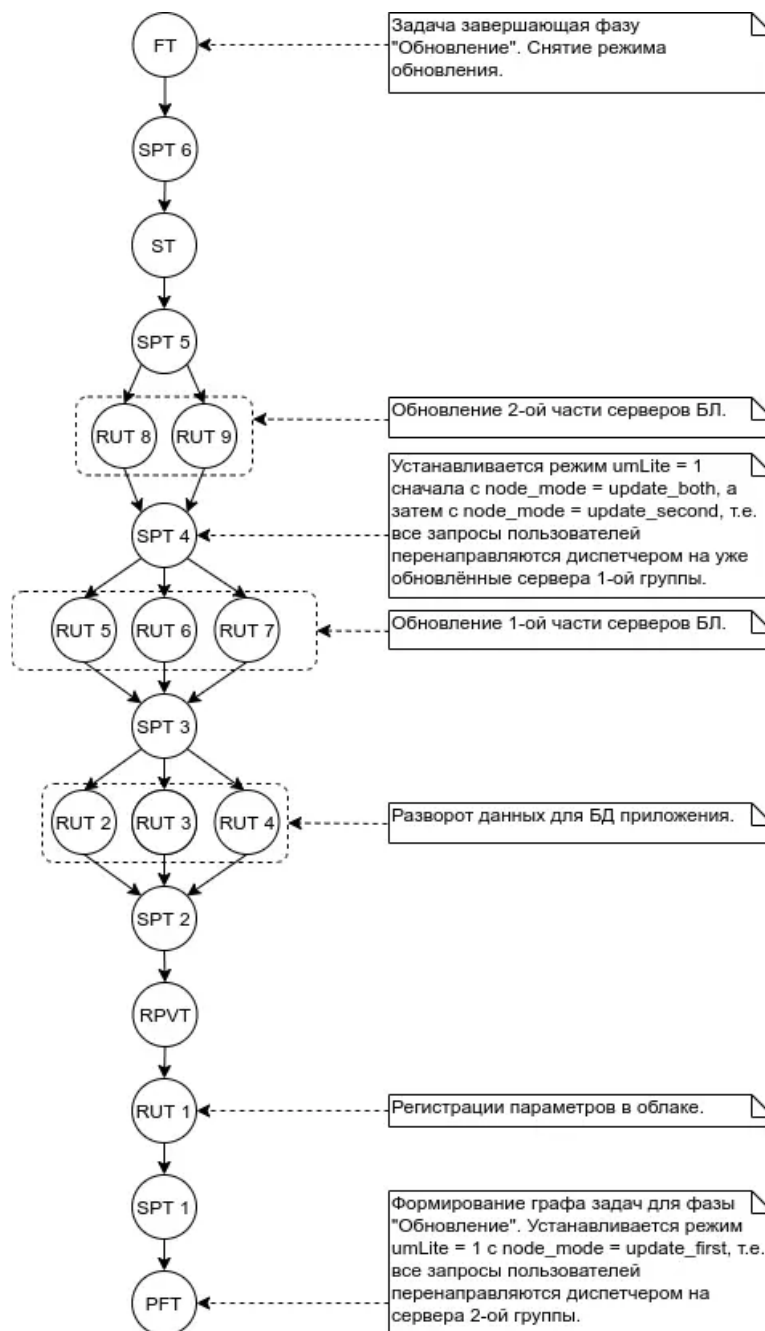


Рисунок 11. Граф задач основной фазы лёгкого обновления

Легкое последовательное обновление

Данный режим обновления предназначен для снижения влияния обновления сервиса на его производительность и, в основном, используется для применения патчей в дневное время. Основное допущение этого режима – изменения между установленной и обновляемой версией должны быть незначительными и совместимыми, чтобы запросы могли попадать как на старую, так и на новую версию сервиса в течении всей фазы обновления. Для запуска обновления в данном режиме необходимо на диалоге запуска обновления (рис. 2) при выборе легкого обновления проставить чек-бокс “Последовательное обновление”.

Во время лёгкого последовательного обновления без участия сервисов представления (СП) игнорируются приоритеты у узлов БЛ, выставляемые управлением облака, которые разделяют узлы на 1 и 2 часть. Режим обновления в данном случае на фазе обновления выставляется один раз, в начале фазы. При вызове метода установки режима передается `mode umSequentialLite = 8`. Установка такого режима никак не влияет на работу диспетчера – для него приложение находится в необновляемом состоянии, но во всем остальном данный режим обновления схож с типом `umLite`, выставляемом при лёгком обновлении.

После установки режима формируются задачи на обновление узлов БЛ, не разделяя на две части, а выстраивая в последовательную цепочку из всех узлов каждого сервиса (группы) (рис. 12, RUT 10-12 и 12-15). При прерывании и возобновлении цепочка продолжает выполняться с прерванного места. При откате прерванного обновления цепочка выстраивается только по тем узлам, которые были затронуты обновлением.

Таким образом, сервис в процессе обновления отдаёт запросы на все доступные узлы сервиса без разбора, а последовательное отключение узлов позволяет проводить обновление незаметно по нагрузке при их достаточном количестве.

При наличии СП среди обновляемых сервисов, логика работы несколько изменяется - узлы БЛ обновляются последовательно по одному, как описано выше, а узлы СП, как и при лёгком обновлении - двумя частями, с маршрутизацией диспетчером клиентских запросов по узлам. Для обеспечения установки режима диспетчера только для СП, в методе установки режима обновления `SetUpdateMode` передаётся `mode umLite = 1` и список СП.

В середине фазы обновления, после установки режима обновления с `node_mode = update_both`, Хоттабыч вычитывает значение параметра конфигурации "Управление версиями.РазбросУведомленийИнтерфейса" и передает его в поле `NotificationRange` события "versionmanager.updatestaticcomplete" (рис. 12, SPT 4). Участок, обрабатывающий событие, при наличии этого поля показывает пользователям уведомление об обновлении в случайное время в переданном диапазоне, что обеспечивает разброс запросов от клиентов в переходе на новые узлы СП. Хоттабыч в этой точке находится в режиме ожидания, а клиентские запросы обрабатываются как обновлёнными, так и старыми узлами. После завершения ожидания, выставляется режим с `node_mode = update_second` и запускается обновление второй части узлов СП.

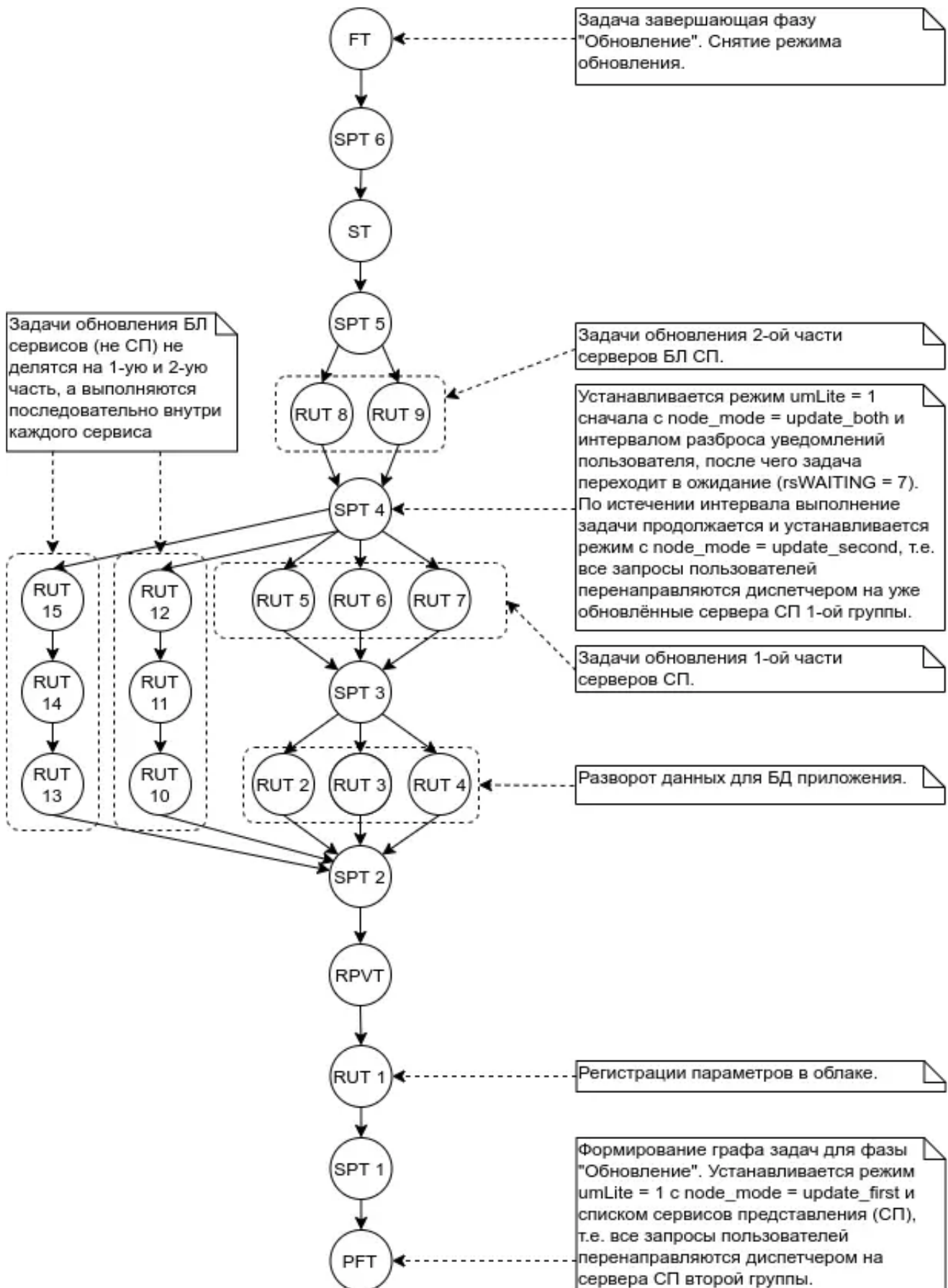


Рисунок 12. Граф задач основной фазы лёгкого последовательного обновления

Полное обновление

Обновление с полной блокировкой и остановкой работы продукта. На протяжении всей основной фазы обновления работа пользователя будет приостановлена, будет отображаться парковочная страница с новостью об обновлении (для чего используется метод облака Диспетчер.УстановитьПарковочнуюСтраницу). В этом типе обновления дополнительно участвуют серверы баз данных, на которых производится конвертация баз. Перед началом обновления происходит блокировка приложения и остановка всех сервисов (рис. 13, PFT). По окончании обновления снимается

режим, который снимает блокировку приложения (рис. 13, FT). Так как во время обновления работа пользователей приостановлена, то появляется возможность обновлять обе части БЛ одновременно (рис. 13, RUT 5-8).

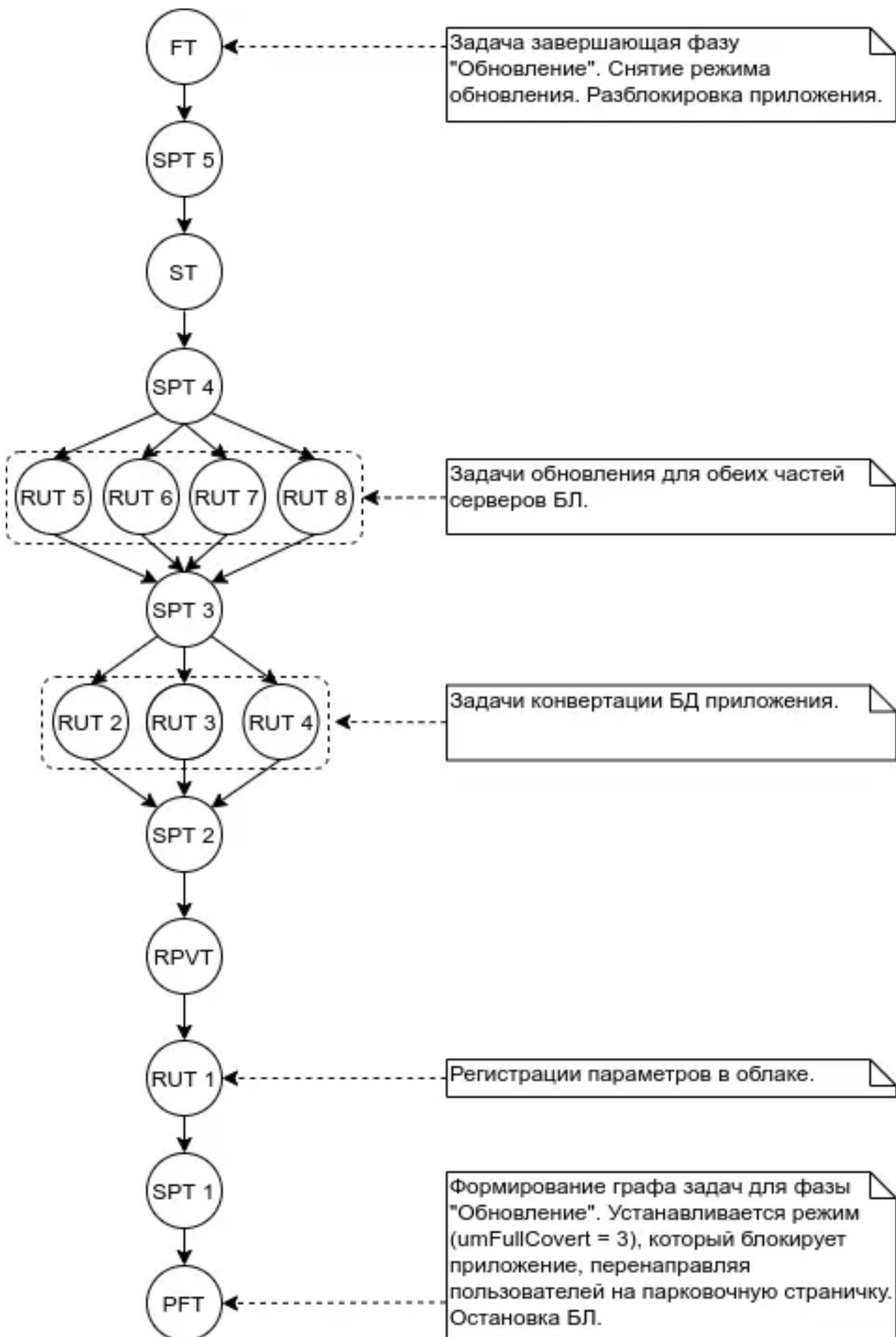


Рисунок 13. Граф задач основной фазы полного обновления

Полное последовательное обновление

Данный режим обновления предназначен для исключения простоя в записи данных во время обновления с конвертацией БД продуктов, у которых есть необходимость непрерывной записи в базу, но для которых является некритичным непродолжительное отсутствие возможности анализа(чтения) данных из базы.

Для запуска полного последовательного обновления необходимо на диалоге запуска обновления (рис. 2) при выборе полного обновления проставить чек-бокс "Последовательное обновление", который отображается только при обновлении продукта, в составе которого есть хотя бы один сервис помеченный в облаке флагом "Разрешено полное последовательное обновление" (можно задать при разработке через genie.saby.ru, установив флаг "Full consistent update allowed" в настройках сервиса (Service customization) на вкладке "Extended"), информация о наличии которых содержится в БД Хоттабыча и актуализируется при каждом опросе агентов приложения. При запуске обновления анализируется результат выполнения CloudService.List для каждого обновляемого сервиса приложения и, если среди записей сервисов есть записи с заданным полем "full_consistent_update", то сервис помечается в БД Хоттабыча т."ИсторияОперацийСервиса" (поле "ЕстьРавноправныеБД", далее в тексте такой сервисбудет обозначаться как сервис непрерывной записи (СНЗ)), и именно к нему будут относиться БД, если БД на этих же хостах указаны и для других сервисов. В диалоге обновления такие группы визуально никак не обособляются.

Алгоритм основной фазы полного последовательного обновления следующий (рис. 15):

- при установке режима обновления БЛ и БД сервиса, помеченного флагом "full_consistent_update", в облаке равномерно распределяются по группам;
- после установки режима обновления в облаке запрашивается информация о том, как именно были распределены сервера БЛ и БД. Эти данные сохраняются в БД Хоттабыча;
- БЛ, относящиеся к СНЗ последовательно по группам перезагружаются — это необходимо, чтобы применить настройки из облака и БЛ обращались исключительно к БД своей же группы;
- все сервисы, кроме СНЗ, блокируются;
- обновление заблокированных сервисов проходит по описанному выше алгоритму полного обновления;
- параллельно идёт обновление каждого СНЗ. Обновление каждого отдельного СНЗ происходит последовательно по группам по следующему алгоритму (рис. 14, на рис.15 группы обозначены как GP 1-3):
 1. сервера БЛ группы выключаются, все запросы пришедшие на данные сервера перенаправляются диспетчером на другие сервера сервиса;
 2. все БД группы конвертируются параллельно;
 3. по завершении конвертации БД, параллельно обновляются все БЛ группы;
 4. группа включается в работу, сервера группы корректно исполняют запросы;
- снятие режима обновления, что приводит к разблокировке прочих сервисов и удалению распределения по группам для БЛ и БД у СНЗ.
- БЛ, относящиеся к СНЗ последовательно по группам перезагружаются — это необходимо, чтобы применить настройки из облака и БЛ обращались ко всем БД сервиса.

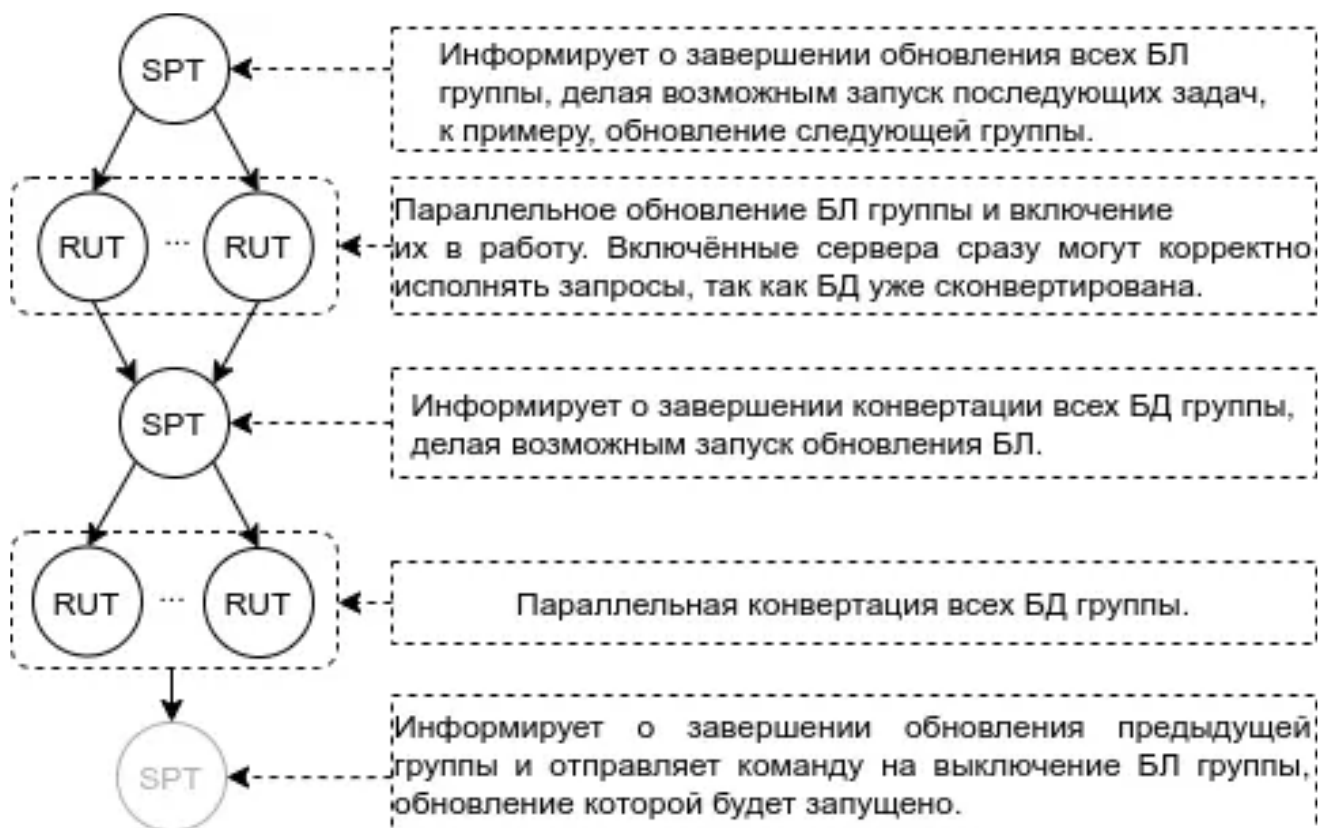


Рисунок 14. Граф задач основной фазы полного последовательного обновления для отдельной группы

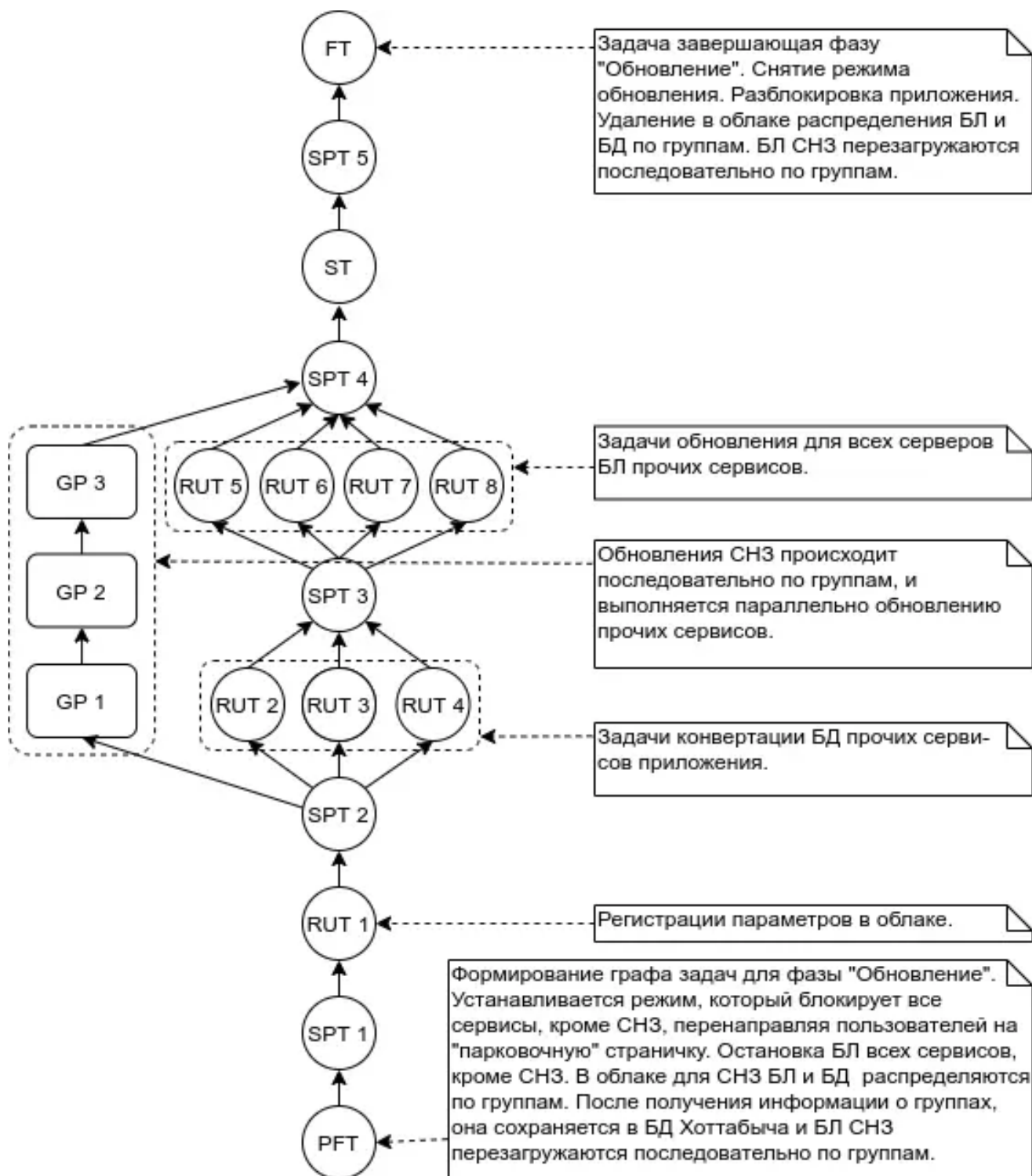


Рисунок 15. Упрощённый граф задач основной фазы полного последовательного обновления

Полное обновление без остановки (посхемное обновление)

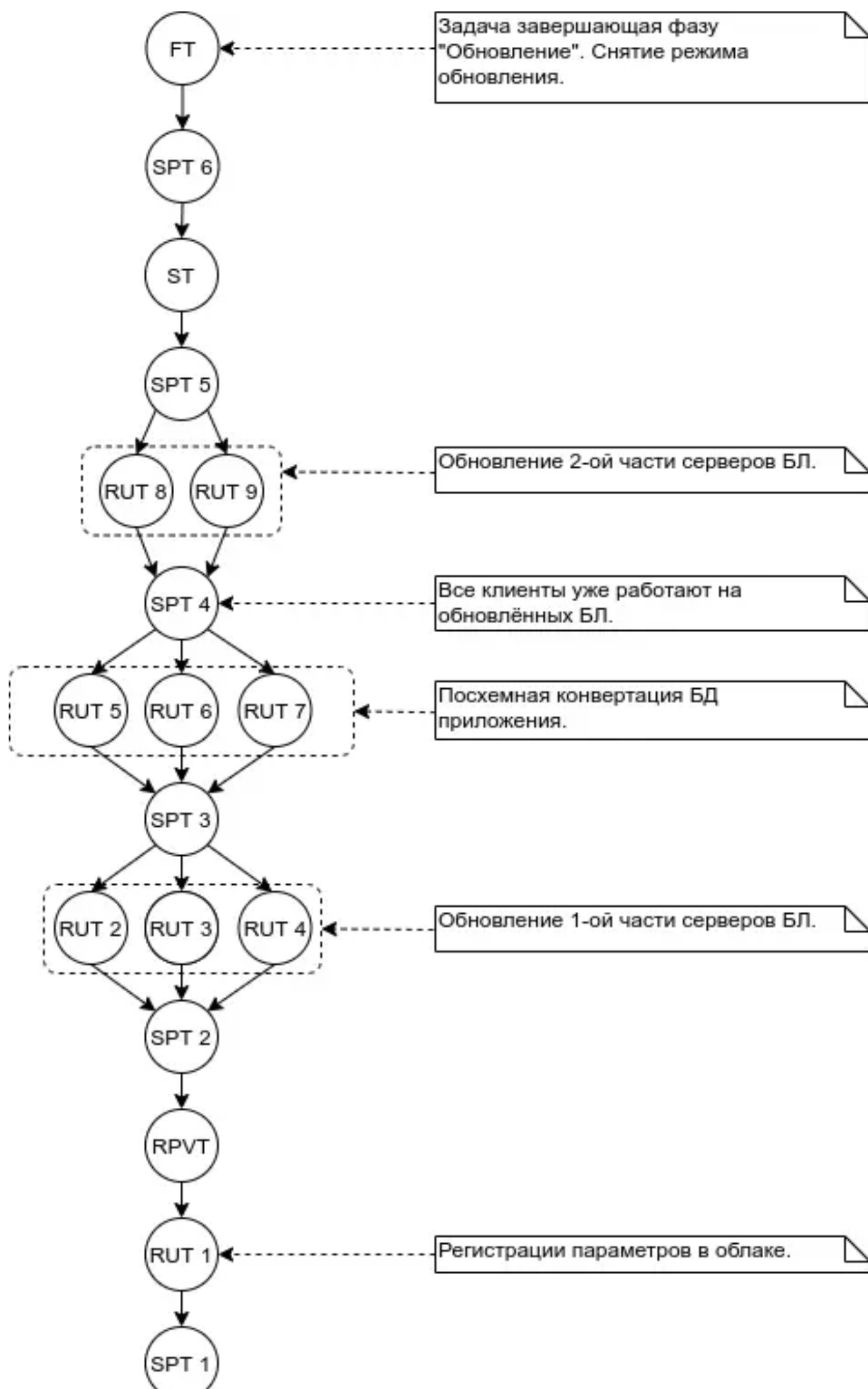
Проходит с конвертацией, но без полной остановки группы (применяется для продуктов онлайн).

На фазе подготовки доставляются дистрибутивы и подготавливаются новые данные в БД – без остановки. На фазе обновления БЛ и статик делятся пополам, половина обновляется (рис.16, RUT 2-3), половина остается на старой версии. Начинается конвертация БД посхемно (рис.16, RUT 5-7): клиент работает на старой БЛ, когда приходит время его конвертации – он блокируется, конвертируется его схема и затем он разблокируется и направляется на новые БЛ. И так процесс идет, пока не будут сконвертированы все схемы. По завершении конвертации все клиенты работают на новых БЛ и тогда вторая группы БЛ и статик обновляется (рис. 16, RUT 8, 9).

Особенностями посхемного обновления являются:

1. возможность задания списков конвертируемых или игнорируемых клиентов в мастере запуска обновления;
2. возможность изменения списка игнорируемых клиентов в процессе обновления для исключения из обновления клиентов, для которых конвертация завершается с ошибкой;
3. возможность выбора обновляемых сервисов и задания списка конвертируемых клиентов в мастере запуска обновления при обновлении на ту же самую версию (по аналогии с лёгким типом обновления). Данный функционал

позволяет "дообновлять" только те сервисы, конвертация на которых завершилась ошибкой.



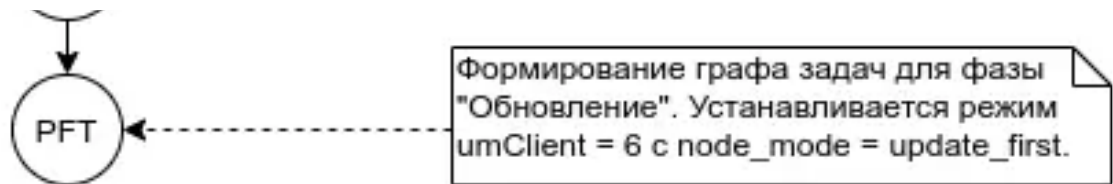


Рисунок 16. Граф задач основной фазы посхемного обновления

Легкая конвертация

Конвертация БД без остановки работы.

Легкая конвертация от полной конвертации отличается следующим:

- продукт не блокируется на время обновления, и БЛ продолжает работать с конвертируемой базой;
- Jinnee не разрывает уже установленные соединения с базой, поэтому во время конвертации возможны ошибки jinnee, связанные с невозможностью наложить блокировку на объект БД;
- некоторые значительные изменения в структуре БД могут блокировать работу пользователей. Например, создание столбца со значением по умолчанию – операция, которая может достаточно на долгое время наложить блокировку на таблицу. В процессе действия этой блокировки запросы к базе от БЛ будут висеть в ожидании.

Для оценки, возможно ли запускать обновление в режиме легкой конвертации, в отчете Хоттабыча "Анализ изменения БД" делается сводка по изменениям с указанием критичности этих изменений для этого типа обновления. Граф задача аналогичен изображённому на рис.11.

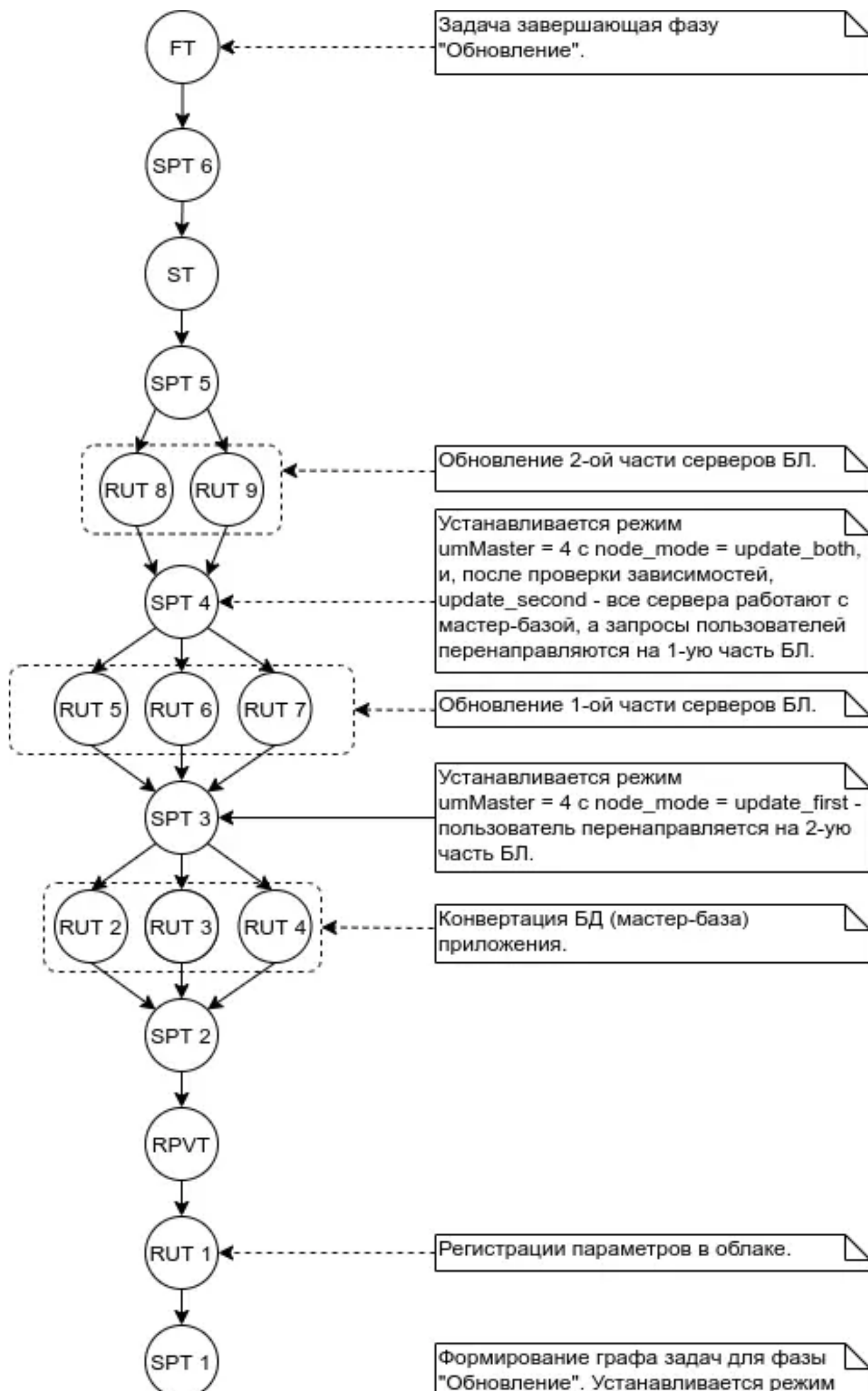
Конвертация мастер-реплика

Мастер-реплика – тип обновления с конвертацией БД без простоя работы сервиса. Для возможности запуска этого обновления у каждой БД продукта должна быть настроена реплика БД, работающая на чтение. Реплика также разгружает мастер-базу, беря на себя часть запросов на чтение.

Алгоритм работы обновления следующий:

1. изначально все серверы БЛ работают как с мастер-базой, так и с репликой;
2. в ходе построения графа задач основной фазы все серверы БЛ переключаются на работу с базой реплики (рис. 17, PFT);
3. отключается репликация баз (рис. 17, PFT);
4. конвертируется мастер-база (рис. 17, RUT 2-4);
5. обновляются серверы БЛ и статик. В процессе обновления серверы получают конфигурацию, которая сообщает серверу работать с мастер-базой. По аналогии с лёгким обновлением происходит установка режима обновления, чтобы перенаправить пользователя на те сервера, которые в данный момент не обновляются;
6. возобновляется репликация, и хоттабыч дожидается момента, когда реплика "догонит" мастер-базу;
7. все серверы БЛ переключаются к исходному состоянию для работы и с мастер базой, и с репликой. Режим обновления снимается на фазе очистки.

Ограничение в работе данного типа обновления следующее: в процессе обновления (в момент выполнения пунктов 2-5) серверы БЛ работают с репликой БД в режиме "только чтение". Это означает, что все запросы на запись в базу не смогут исполниться и будут выдавать ошибку.



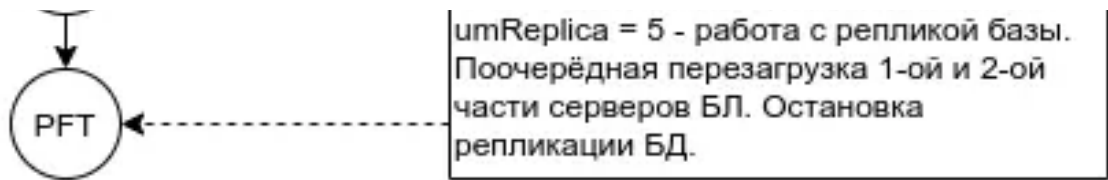
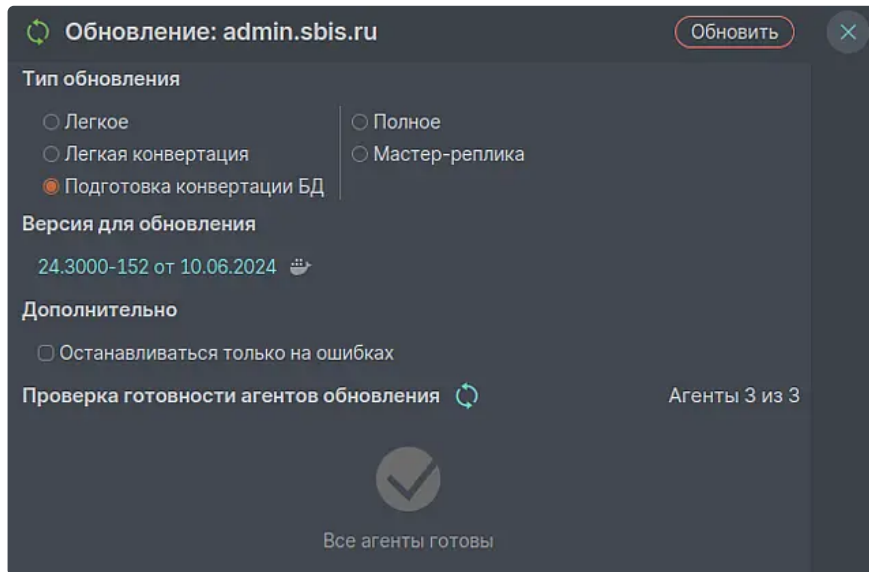


Рисунок 17. Граф задач основной фазы обновления типа мастер-реплика

Подготовка конвертации БД

Данный тип обновления по своей сути является разделением обновления с конвертацией на два отдельных обновления: первое обновление проводит подготовку конвертации базы данных (это возможно ввиду того, что алгоритм проведения подготовки не отличается на разных типах обновления), а второе обновление с конвертацией проводит полноценное обновление согласно выбранному типу, минуя проведение конвертации базы на подготовке.

При запуске подготовки конвертации БД пользователь выбирает новый одноименный тип обновления:



Этот тип передается в метод запуска обновления. При получении из облака списка сервисов и узлов, Хоттабыч отбрасывает всё, кроме узлов баз данных; в том числе в состав обновляемого приложения не добавляются узлы статики и cdn, а также логики регистрации метаданных. Так как логики баз данных - единственные составляющие обновления, для них фаза подготовки проводится в обычном режиме. Единственное отличие - в утилиту `jinnee_lite` передается параметр, изменяющий время жизни хэша схем - `Управление версиями.Конвертеры.ВремяХраненияХэшаСхем`. Этот параметр задается в настройках облака.

После окончания фазы подготовки, фазы обновления и подтверждения пропускаются и обновление финализируется. Итогом проведения подготовки является то, что в базах данных прикладного приложения внесены некоторые изменения из новой версии (созданы новые таблицы, добавлены индексы и т.п.), но при этом версия БЛ не изменилась. Также было проведено хэширование схем с удлиненным временем хранения хэша, что при проведении основного обновления позволит значительно снизить нагрузку.

Далее, через какое-то время, например, через сутки, запускается основное обновление с конвертацией. Это обновление может быть любого типа: легкая конвертация, посхемное, полное, мастер-реплика. Любой запуск обновления с конвертацией анализирует историю обновлений приложения в Хоттабыче и смотрит, не было ли перед этим запуска обновления с типом "Подготовка конвертации БД". Если не было - обновления проходит в обычном режиме, если было - Хоттабыч проверяет, не истекло ли еще время хранения хэшей. И только если хэши все еще актуальны, обновление запускается в особом режиме, в котором на фазе подготовки пропускается обработка баз данных (она была сделана в предыдущем обновлении). На следующих фазах обновления, чтобы утилита `jinnee_lite` переиспользовала сохраненные хэши, Хоттабыч этой утилите подменяет идентификатор обновления, забирая его из предыдущего обновления, проводившего подготовку. Таким образом джинн думает, что он просто продолжает предыдущую конвертацию.

Если по какой-то причине "основное" обновление задержалось и время жизни хэшей истекло - в этом ничего страшного нет. При запуске основной фазы без наличия хэшей джинн построит для схем свой план конвертации.



Может быть следующая ситуация: провели подготовку конвертации в один день, а на другой день, когда должно проходить основное обновление, приходит критичная доброска в дистрибутив с правкой конвертации. Таким образом хэши схем становятся невалидными. Но Хоттабыч об этих правках в дистрибутиве ничего не знает. Для этого на диалоге запуска добавили дополнительный флаг "Принудительно запустить подготовку конвертации". Его установка указывает обновлению проигнорировать данные из истории о проведенной подготовке и провести её заново.

Параллельное обновление приложений, использующих общие базы

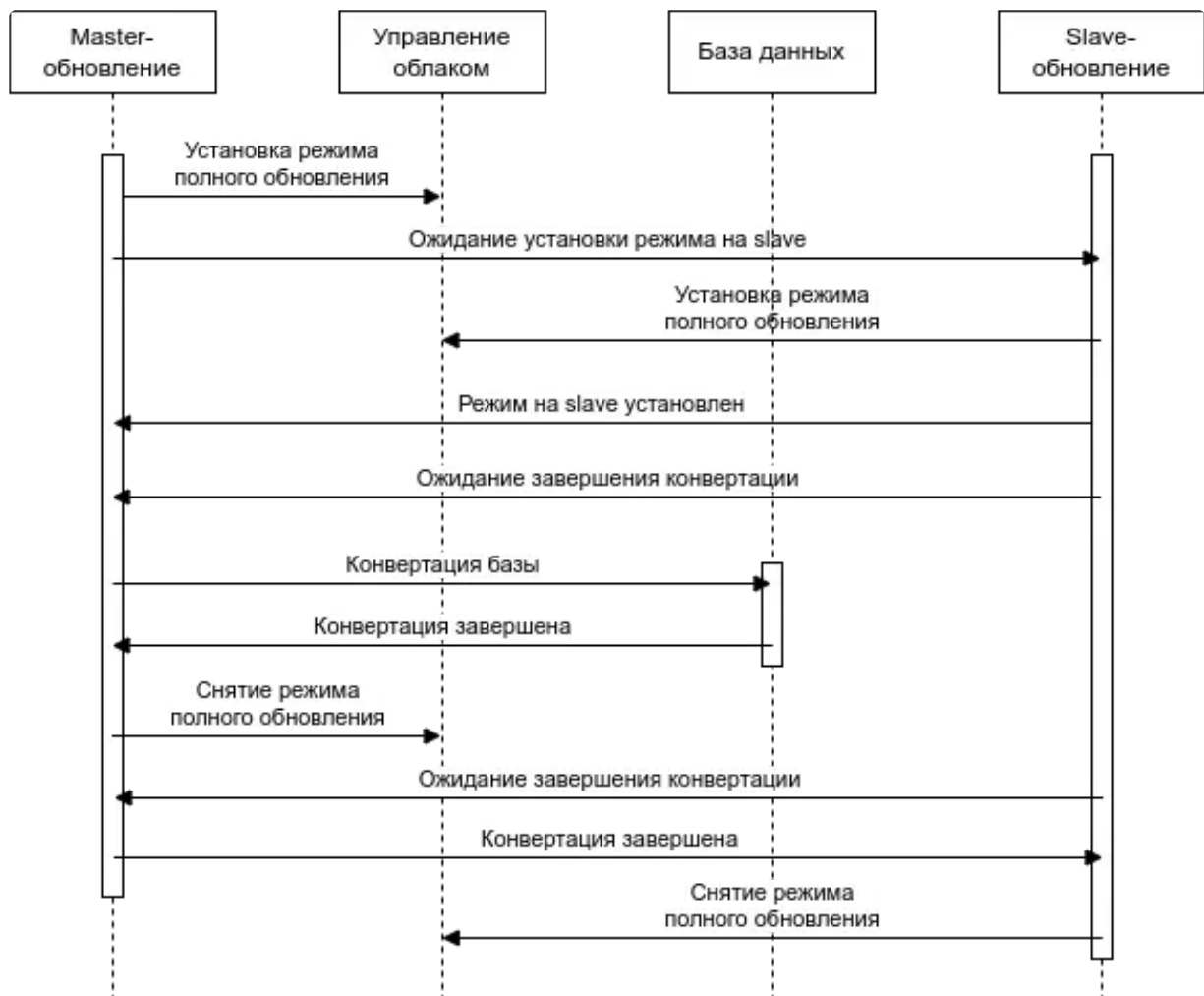
Функционал позволяет запускать и контролировать ход обновления сразу нескольких приложений, сервисы которых используют одну и ту же базу. Запуская обновляться такое приложение, Хоттабыч запрашивает у облака список других приложений, которые используют ту же базу. Согласно полученному списку, сразу после запуска основного приложения, происходит запуск всех связанных приложений. В зависимости от типа запускаемого обновления, различается и схема поведения этих обновлений. Общим является то, что непосредственно конвертацией занимается главное, master-обновление, а все slave-обновления в точке конвертации базы ожидают выполнения действий на master-приложении. Также существуют и другие точки синхронизации, которые опишем ниже.

Легкая конвертация



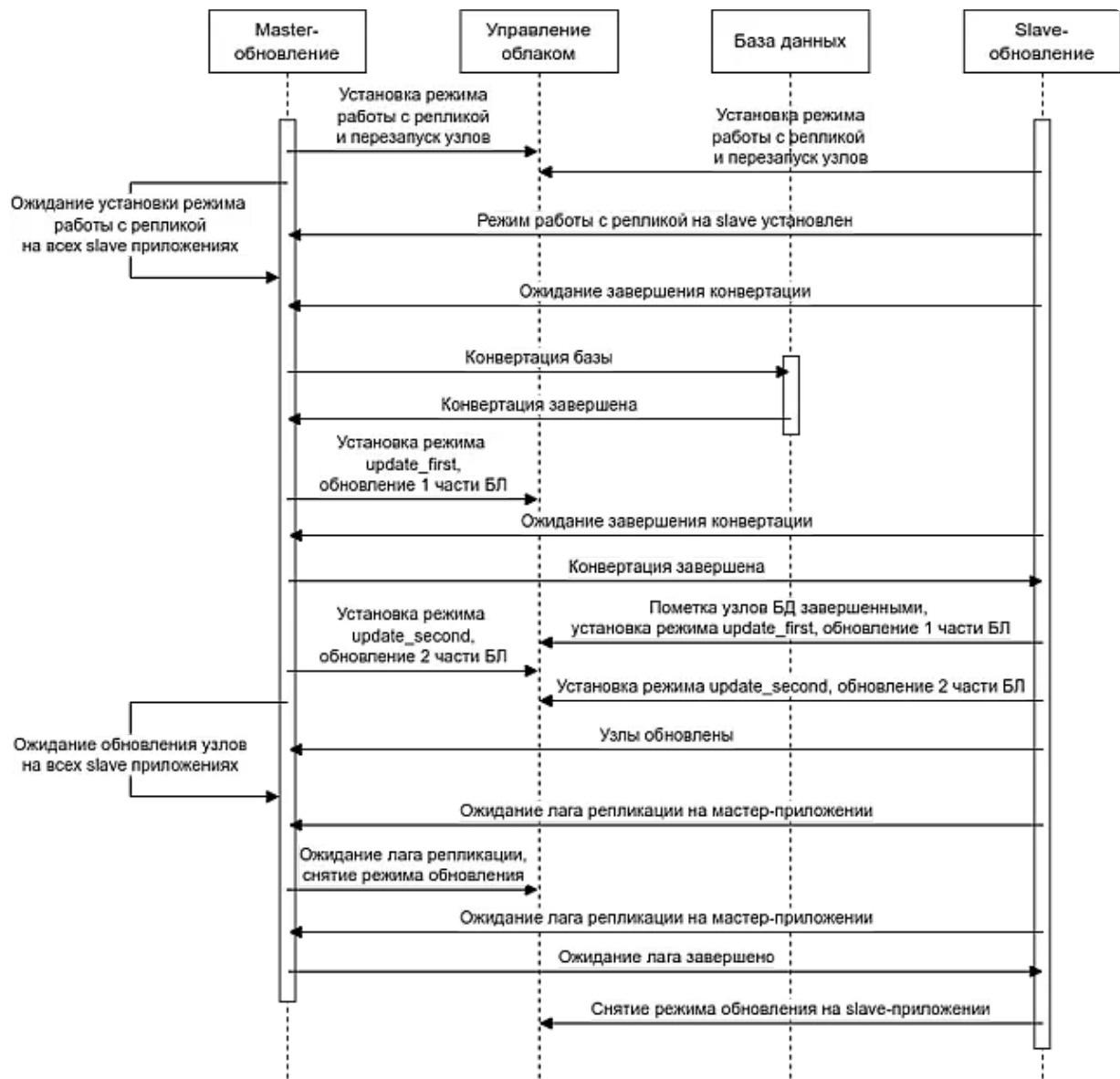
1. Master-приложение проводит конвертацию БД и устанавливает признак завершения конвертации master-базы и продолжает обновление в обычном режиме, никого не ожидая.
2. Slave-приложение дожидается установки признака завершения master-базы, помечает у себя узлы БД выполненными и продолжает обновление в обычном режиме.

Полное обновление



1. Master-приложение устанавливает режим полного обновления. И встает в ожидание установки режима обновления всех slave-приложений.
2. Slave-приложения устанавливают режим полного обновления и встают в ожидание окончания конвертации master-базы master-приложения.
3. Master-приложение дожидается окончания установки режимов на всех slave-приложениях и после этого выполняет конвертацию баз данных.
4. Master-приложение устанавливает признак, что конвертация мастер-базы завершена и далее продолжает обновление более никого не ожидая.
5. Slave-приложения вычитывают признак, что master-приложение закончило конвертацию БД, помечают обновление собственной БД выполненной и далее продолжают обновление в обычном режиме.

Мастер-реплика



1. Master-приложение устанавливает режим обновления работы со slave-базой, выполняет перезапуск узлов БД и встает в ожидание установки режима обновления всех slave-приложений.
2. Slave-приложения устанавливают режим обновления работы со slave-базой, перезапускают узлы БД и встают в ожидание окончания конвертации master-базы master-приложения.
3. Master-приложение дожидается окончания установки режимов на всех slave-приложениях, после этого выполняет конвертацию баз данных.
4. Master-приложение устанавливает признак, что конвертация мастер-базы завершена, устанавливает режим обновления работы только с master-базой, перезапускает узлы и встает в ожидание перезапуска всех slave-приложений.
5. Slave-приложения вычитывают признак, что master-приложение закончило конвертацию БД, помечают обновление собственной БД выполненной, устанавливают режим обновления работы только с master-базой, перезапускают узлы и встают в ожидание окончания репликации slave-базы.
6. Master-приложение дожидается выполнения предыдущего пункта на всех slave-приложениях, включает репликацию, дожидается нормализации лага и устанавливает признак, что slave-база тоже актуальна. После этого продолжает обновление в обычном режиме.
7. Slave-приложение вычитывает флаг готовности slave-базы и тоже продолжает выполнение своего обновления в обычном режиме.

Уведомления об обновлениях в пользовательском интерфейсе

[Стандарт](#)

Подключение

В случае если ваше приложение строится на технологии [SabyPage](#), то дополнительно ничего делать не нужно.

В противном случае необходимо в afterMount точки входа в ваше приложения импортировать модуль [ServiceUpdateNotifier/notifier:PopupController](#) и вызвать у него метод init.

```

1 class Index extends Control {
2     ...
3     protected _afterMount(): void {
4         import('ServiceUpdateNotifier/notifier').then(({PopupController})
=> {
5         PopupController.init();
6     })
7     }
8 }

```

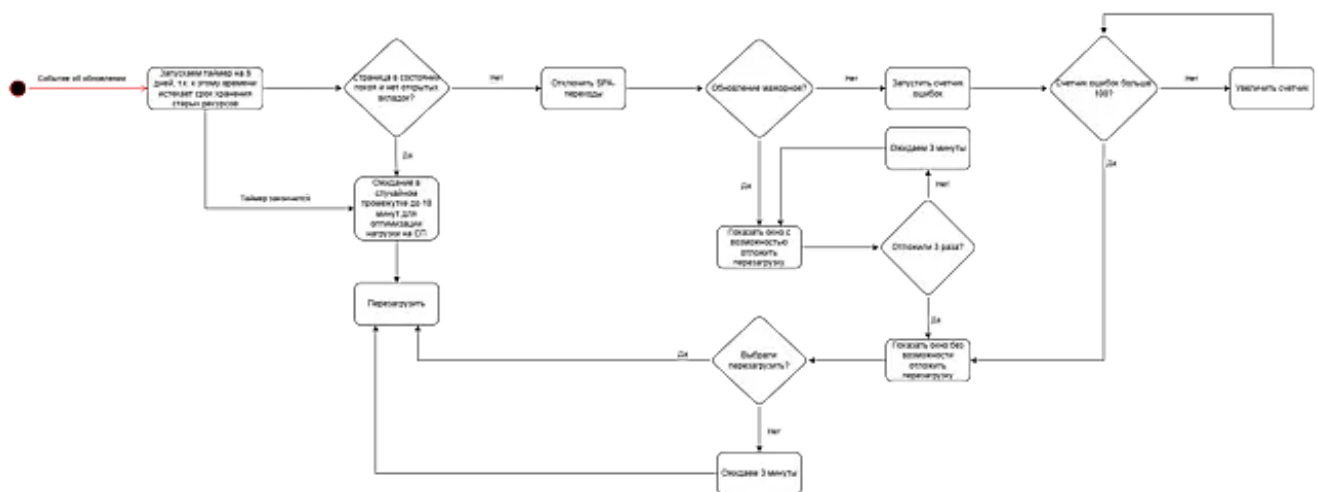
Получение версии и проверка актуальности сайта

Для получения версии сайта используйте метод [ServiceUpdateNotifier/notifier.getVersion](#), который вернет объект с актуальной версией.

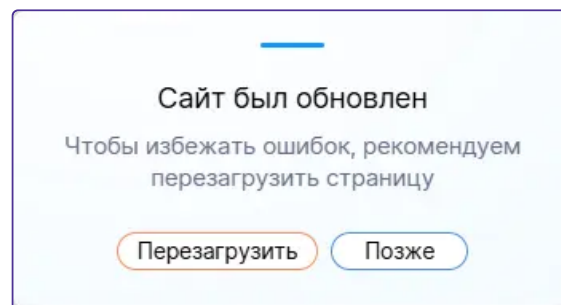
Для проверки актуальности версии сайта используйте метод [ServiceUpdateNotifier/notifier.isVersionOutdated](#), который вернет **true**, если версия устарела.

Схема уведомления пользователей

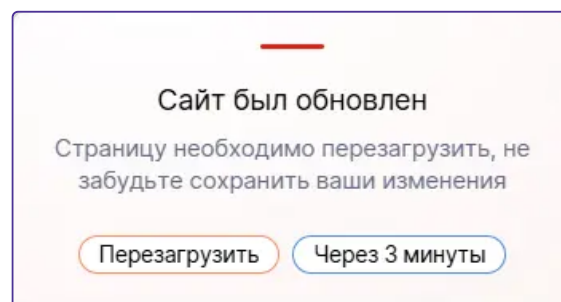
Уведомления пользователей выполняется по следующей схеме:



После получения события о мажорном обновлении пользователю показывается окно, предлагающее обновить страницу для применения обновления:



В случае, если пользователь откладывает перезагрузку 3 раза или счетчик ошибок на странице достиг 100, то появляется окно без возможности отложить перезагрузку:



При выборе варианта "Через 3 минуты" будет запущен таймер, информирующий пользователя об оставшемся времени до принудительной перезагрузки страницы:



Страница перезагрузится через 03:00 X

После истечения таймера страница будет автоматически перезагружена.

Применение патчей на дистрибутивы

Применение патча - это замена указанных файлов в дистрибутивах сервисов. Исправленные файлы передаются Хоттабычу, он заменяет файлы в существующем дистрибутиве и модифицирует его версию, и в итоге выкладывает файлы дистрибутива на сервис. Всё происходит очень быстро!

Патчи применяются в том случае, когда в дистрибутиве требуется срочно заменить один-два файла, а сборка нового дистрибутива нецелесообразна, например её создать невозможно или требуется слишком много усилий или времени.

При применении патча создается копия дистрибутива в Хранилище дистрибутивов, при этом реального копирования файлов не происходит, а создаются только ссылки на уже существующие файлы, поэтому это происходит быстро.

Хоттабыч скачивает оригинальный дистрибутив и патч на подключенный диск (test-fs), распаковывает архивы, заменяет файлы дистрибутива файлами из патча, упаковывает архивы, заменяет архивы в копии дистрибутива, переименовывает дистрибутив, добавляя номер патча к версии. Таким образом получается новый дистрибутив, с внесенными изменениями.

Затем приложение обновляется на дистрибутив с патчем.

Сборка интерфейсных патчей

Для jippee есть реализация провайдера сборки интерфейсных патчей. На входе он получает измененные разработчиком модули (а также окружение с зависимыми модулями), а на выходе - готовый zip-архив только с измененными модулями.

Архитектура провайдеров сборки хорошо ложится на функционал сборки интерфейсных патчей:

1. FilterModules() - отфильтровывает те модули, которые пришли на вход для сборки патча.
2. CalculateVersion() - вычисляет версии модулей согласно общим алгоритмам всех провайдеров сборки.
3. Upload() - по результату сборки модулей выкачивает исходные модули, сравнивает их, строит пофайловую разницу и упаковывает в zip.

В jenkins есть сборка интерфейсных патчей с возможностью переключения сборки патча по-новому и на сборку полного дистрибутива. На выходе сборка выдает в артефактах несколько архивов патча по каждому собираемому дистрибутиву. Это обусловлено тем, что в общем случае один и тот же патч может по-разному накладываться на каждый из дистрибутивов. Архивы патчей именуются по имени дистрибутива и хэшу их данных. Если хэши одинаковые - можно смело брать любой из архивов и прикреплять к ВНР. Если архивы разные - сборка падает с ошибкой, в этом случае нужно разобраться какая ситуация привела к сборке дистрибутивов с различными модулями.

Из-за специфики структуры интерфейса при изменении модуля интерфейса (и соответственно его версии) должны измениться корневые файлы в папке resources (contents.json, другие справочники и их упакованные и минифицированные версии). Для их модификации метод применения патча вызывает джинн, который по измененным модулям их перестраивает.

Применение патчей

Процедура применения патчей одинакова как для интерфейсных патчей, так и для патчей БЛ.

Дистрибутив состоит из множества zip-архивов, поэтому задача применения патча сводится к

1. Определить какой архив нужно патчить.
2. Замене файлов в архиве.
3. Загрузке архива в хранилище.

Определение архива для применения патча

Дистрибутив имеет модульную структуру, а хранится каждый модуль в отдельном архиве. Во время применения патча меняется только один или несколько модулей. В патче хранятся только измененные файлы, по пути до файлов можно определить название модуля и тип (бизнес-логика или интерфейс) модуля. Поэтому скачиваются только измененные модули, а неизменённые - нет.

Замена файлов в архиве

После определения и скачивания нужных для патча модулей, требуется изменить скачанные модули.

Для того, чтобы не нужно было полностью распаковывать и запаковывать архив используется бесплатный открытый архиватор 7zip. Он поддерживает операции удаления файла из архива. Составляется список файлов для удаления из архива по набору файлов в патче, и с помощью 7zip старые файлы удаляются из архива. Затем новые файлы добавляются в архив.

Загрузке дистрибутива в хранилище

Создается копия дистрибутива, со ссылками на файлы старого дистрибутива. Затем измененные архивы загружаются в хранилище и заменяют старые файлы.

Обновление сборки дистрибутива

Так как при применении дистрибутива меняется номер сборки, то также патчится .s3distr конфиг дистрибутива, находящийся в корневых архивах дистрибутива.

Можно применять патч на

- Файлы бизнес-логики, для которых не требуется компиляция: *.dicx, *.orx, *.cnv, *.py.
- Файлы статики: *.js, *.tmpl, *.css, картинки (*.png, *.jpg и т.д.), текстовые файлы и т.п.

Нельзя применять патч на

- Патч не исправляет бинарные файлы, требующие бинарной совместимости с другими файлами дистрибутива.
- Патчем нельзя изменить закешированные веб-браузером файлы, так как версия файлов не меняется.

Развёртывание web-приложений

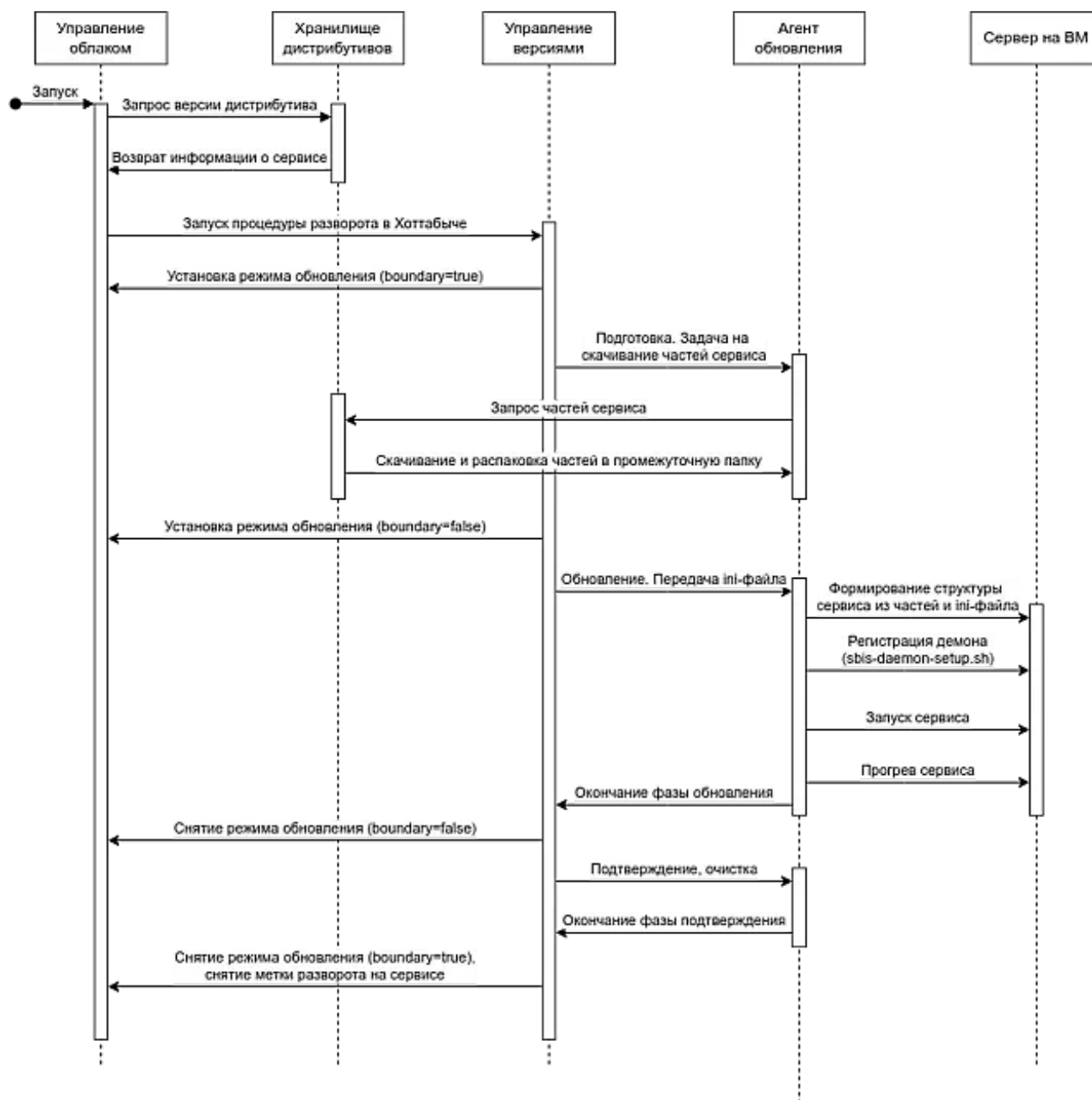
Процесс развёртывания приложения запускается из облака.

Проверка возможности установки сервиса на сервере запрашивается у агента обновления метод UpdateAgent.CheckServiceInstalled.

Метод Хоттабыча Продукт.Install запускает процесс развёртывания приложения в Хоттабыче. В Продукт.Install кроме прочих параметров приходит RecordSet, содержащий daemon_name и параметры установки для заданных БЛ.

В ходе развёртывания web-приложений строится граф, аналогичный лёгкому обновлению (рис. 10), но никаких режимов не устанавливается, так как либо приложение ещё не установлено, либо развёртывается новый сервис, и этот процесс не должен мешать работе всего приложения. А если в аргументах метода Продукт.Install был задан параметр lite_install = true, то к обновлению не будут добавлены логики для регистрации параметров, кластеров и задач планировщика, а, следовательно, и задач для них не будут построены.

Алгоритм разворота нового сервиса на сервере виртуальной машины:



Разворот сервиса в Kubernetes значительно проще: вместо агента управления в процессе разворота управление версиями по API взаимодействует с сервисом Datacenter, передавая ему ссылку на образ, сформированный ini-файл, а также параметры для обратного вызова.

Необлачные приложения

Необлачными приложениями в Хоттабыче называются приложения, не описанные в структуре облака и загружаемые на update.sbis.ru. Для обновления экземпляр необлачного приложения самостоятельно обращается на сайт update.sbis.ru и забирает необходимые ему файлы.

Сайт update.sbis.ru архитектурно состоит из мастер-сервера, содержащего эталонные, загруженные в него данные; набора слайвов, возможно географически распределенных для более быстрой раздачи клиентам; сервиса-балансировщика, в задачи которого в том числе входит функционал синхронизации данных между мастером и репликами.

В облаке описывается url сервиса балансировщика, через который предоставляется REST API управления его функционалом. В Хоттабыче задается связь сервиса балансировщика и облака, в котором запускается обновление необлачного приложения. В Хоттабыче заводятся необлачные приложения, запуск которых будет инициировать загрузку данных на update.sbis.ru.

Процесс загрузки определяет мастер-сервер балансировщика и через агента обновления, установленного на нем, доставляет заливаемый дистрибутив. На фазе обновления утилита агента подменяет данные в папке мастер-сервера и инициирует через API балансировщика процедуру синхронизации зеркал. Агент дожидается окончания синхронизации, что сигнализирует об окончании фазы обновления. На подтверждении агент удаляет бэкапы выгрузки и остальные временные файлы.

Реализация на БЛ Хоттабыча

Особенностей реализации функционала на стороне БЛ Хоттабыча практически нет и загрузка мало чем отличается об обычных обновлений. Хоттабыч получает из облака адрес API-сервиса балансировщика, осуществляет на него get-запрос `/status/`, который возвращает информацию о мастер-сервере и базовом пути загрузки. В дальнейшем БЛ Хоттабыча в обычном режиме взаимодействует с агентом.

Реализация на агенте обновления

1. Доставка дистрибутива. Тут все происходит так же, как в обычных обновлениях: скачивается утилита из ХД, потом утилите передается ссылка на zip с дистрибутивов, который выкачивается во временную папку мастер-сервера.

2. Основная фаза.

На основной фазе в первую очередь нужно убедиться, что никакая другая загрузка не взаимодействует с тем же путем загрузки. Для этого от доп. пути загрузки отделяется папка первого уровня и в ней проверяется наличие lock-файла, который мог быть создан другой выгрузкой или даже другим Хоттабычем. Если lock-файла нет, утилита создает свой файл. Логика блокировки описана в файле `upload_locker.cpp`.

Следующим шагом требуется убедиться, что балансировщик в данный момент не синхронизирует папку. Это делается запросом `GET /api/sync/id` к балансировщику, где `id` – имя папки первого уровня. Если синхронизация уже запущена – падаем с ошибкой.

Далее осуществляем подмену файлов на мастер-сервере. В зависимости от типа выгрузки делаем либо перемещение, либо копирование с заменой.

После актуализации данных на мастер-сервере, запускаем через балансировщик синхронизацию зеркал: `POST /api/sync/id`. Далее периодически проверяем статус синхронизации, читая процент синка через `GET /api/sync/id` и передавая его через агента в журнал Хоттабыча. После завершения синхронизации утилита удаляет lock-файл и завершает работу на основной фазе.

3. Подтверждение. На фазе подтверждения удаляются: бэкапы выгрузки при обновлении с заменой, удаляется временные файлы, логи и утилита.

Сценарии обновления необлачных приложений

В процессе развития обновления необлачных приложений, появилась потребность управлять последовательностью обновления клиентских приложений, описывая в Хоттабыче сценарии выкладки. Сценарий обновления – это некоторый набор правил обновления, который задается в Хоттабыче, с процессом выкладки доставляемый на update.sbis.ru и получаемый клиентским кодом для принятия решения, можно ли производить обновление.

Создание сценариев обновления

Сценарии создаются в интерфейсе Хоттабыча и сохраняются индивидуально для каждого облака и для каждого необлачного приложения в них, никаким образом не пересекаясь и не синхронизируясь. Это означает, что сценарий, созданный для конкретного приложения, например, на `test`, должен быть также вручную заведен и в остальных облаках, в том числе и на рабочем, через ВНР.

Интерфейс создания редактирования сценариев расположен на диалоге запуска необлачного приложения.


Загрузка: Sbis3Plugin

Загрузить

✕

Параметры загрузки

Выберите сценарий:

Сценарий не выбран

+

Вид загрузки:

С заменой

Путь загрузки:

Sbis3Plugin/21.6225.65

Версия для загрузки

21.6225.1-65 от 18.01.2022

☐ Останавливаться только на ошибках

Проверка готовности агентов обновления

↻

Агенты 1 из 1

✓

Все агенты готовы

У каждого сценария есть имя - текстовое поле, по которому связывается сценарий с планами выпуска. Для того, чтобы выкладку приложения запускать по одному и тому же сценарию на разных стендах, этот сценарий должен иметь одинаковое имя.

Сценарий - это набор правил, каждое из которых характеризуется следующими полями:

- Тип правила. Возможные типы: "Идентификатор пользователя", "Идентификатор клиента", "Идентификатор устройства", "Псевдоним версии", "Тип приложения", "Расширение", "Интервал хранения предыдущей версии", "Для всех". Назначение каждого из типов подробно описано в [общей документации обновления оффлайн приложений](#).
- Значение правила. Для задания в качестве значения массива необходимо чтобы переменные в массиве были в не кавычек. В качестве разделителей необходимо использовать запятые или знак точки с запятой. Значение правил сохраняется в БД Хоттабыча в строковом представлении.
- Комментарий. Описательное поле, объясняющее назначение правила.
- Время действия. Время можно задать в двух вариантах: либо конкретную дату/время, либо интервал в часах с момента выкладки.

Выпуск приложений по сценариям

Как уже было сказано выше, работа в планах выпуска содержит только имя запускаемого сценария выкладки. Если сценарий не выбран, или имя сценария не задано на работе плана выпуска, то выкладка происходит по старой схеме. План выпуска при запуске работы в качестве аргументов запуска обновления в том числе передают и имя выбранного сценария.

Хоттабыч на основе выбранного сценария формирует json-объект с описанием поддерживаемых на текущий момент версий (у которых не истекло время жизни) и правил, на основе которых эти версии выкладывались. Вот пример такого объекта:

```

1  {
2    "21.6225.69": {
3      "released": "2022-01-27 16:49:13",
4      "rules": [
5        {
6          "restriction": "all",
7          "rule_timeout_rel": 0,
8          "values": [
9            ""
10         ]
11       }

```



```

12     ],
13     "status":"active"
14 },
15 "22.1200.410":{
16     "released":"2022-01-27 13:36:59",
17     "rules":[
18         {
19             "restriction":"all",
20             "rule_timeout_rel":0,
21             "values":[
22                 "all"
23             ]
24         }
25     ],
26     "status":"active"
27 },
28 "22.1200.415":{
29     "released":"2022-01-28 11:44:28",
30     "rules":[
31         {
32             "restriction":"all",
33             "rule_timeout_rel":0,
34             "values":[
35                 "all"
36             ]
37         }
38     ],
39     "status":"active"
40 }
41 }

```

В процессе выкладки файлов на мастер-сервер update.sbis.ru, хоттабыч передает утилите обновления json с этим объектом, который сохраняется в папку первого уровня выкладываемого приложения. В случае отката обновления, старый json-файл восстанавливается из бэкапа.

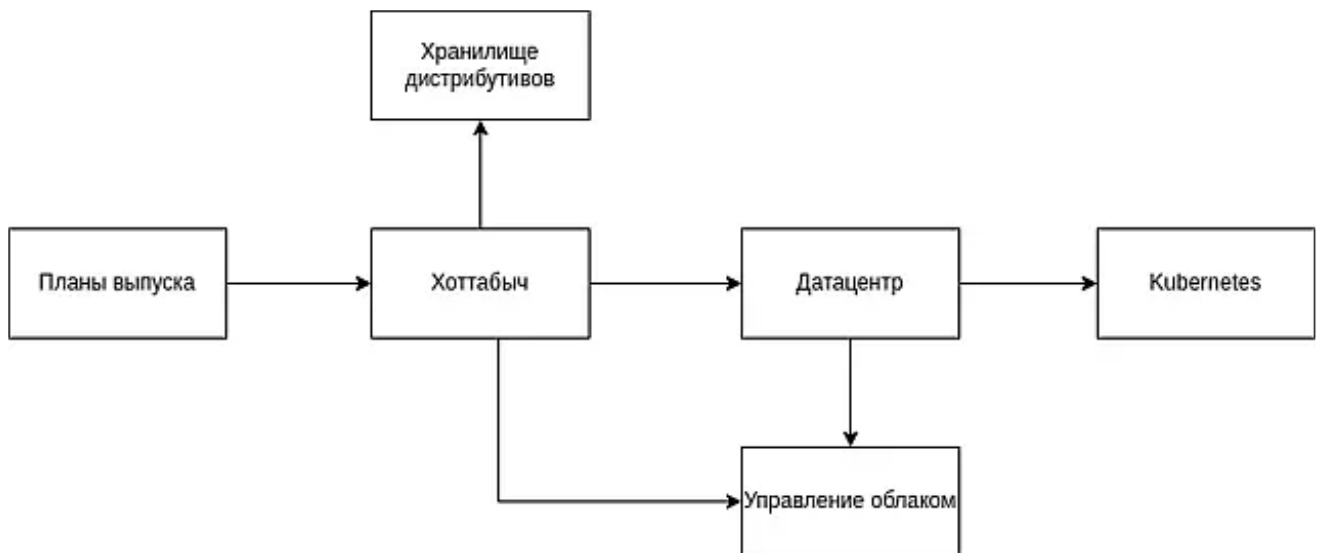
Остановка и откат выкладки

После выкладки необлачного приложения со сценарием, после фазы подтверждения обновления запускается особая фаза применения правил, недоступная и не показываемая для других типов обновлений. Во время прохождения этой фазы сам Хоттабыч уже ничего с этим обновлением не делает. Суть этой фазы - дождаться окончания времени жизни версии и дать возможность сотруднику эксплуатации приостановить или откатить выкладку, если что-то пошло не так.

Все обновления, находящиеся в этой статье, доступны для обозрения и управления через список истории обновлений по приложению. Во всех таких обновлениях мы видим незавершенный индикатор обновления. Тем не менее, данная фаза не мешает запускать новые выкладки по приложению.

Механика выполнения остановки и отката выкладки на фазе применения правил схожа в терминах Хоттабыча. При выполнении этих команд, Хоттабыч формирует актуальную json-конфигурацию выкладки, заменяя поле status на "pause" и "rollback" соответственно. Затем json передается на мастер-сервер update.sbis.ru и запускается синхронизация зеркал. Дальнейшая логика паузы и отката уже заложена в самих клиентских приложениях. После постановки статуса на "pause" сохраняется возможность возобновить выкладку, при которой статус конфигурации меняется на "active".

Обновление кластеров



Связи сервисов, участвующих в обновлении

Последовательность вызовов при обновлении:



Алгоритм последовательности действий при обновлении

Хоттабыч

В интерфейсе Хоттабыча реализован новый справочник, аналогичный справочнику приложений - справочник кластеров. Справочник строится на основе данных, возвращаемых API-методом облака со списком кластеров. Справочник иерархический и содержит в себе уровни с типами кластеров, с папками (уровней папок может быть много) и конечными экземплярами кластеров.

Запуск обновления кластеров доступен лишь на папках. Т.е. запустить обновление единичного кластера из структуры кластеров не получится, для этого придется выбрать для запуска папку с кластерами, а на диалоге запуска обновления выбрать те кластеры из папки, которые будут участвовать в обновлении. Метод запуска обновления формирует структуру истории операций, где в истории операций продукта записывается идентификатор обновления, в истории операций сервиса - группировка по разделам (папкам), в истории операций логики - конечные экземпляры кластеров. Тут стоит понимать, что конечные экземпляры кластеров - это еще не сами обновляемые серверы, это может быть несколько серверов. Тем не менее для Хоттабыча этот экземпляр кластера будет служить единицей обновления.

Для поддержания общности с другими типами обновления, в обновлениях кластеров так же будут присутствовать все фазы обновления, хотя по факту использоваться будут только 2 - обновление и откат. Другие фазы просто не будут выполнять никаких действий.

Кластеры в управлении облаком

На данный момент справочник кластеров в облаке уже умеет описывать кластеры, развернутые в Kubernetes. Для работы обновлений были внесены следующие их доработки:

1. Описание параметров и их значений для формирования helm-чарта. Управление облаком знает структуру формирования helm-чарта, поэтому пользователю отдает для редактирования плоский список лишь тех параметров, которые нужно задавать вручную. Все автоматически генерируемые параметры облако подставляет само. В нижней части окна с настройками helm показывается предпросмотр чарта в том виде, в котором он будет передан в Kubernetes.
2. API для Хоттабыча. Хоттабычу для обновления контейнеров не требуется каких-либо сведений о конечных серверах кластеров. Требуется лишь общая информация о доступных в облаке типах кластеров, их папок и экземпляров кластеров. Все эти данные можно отдать в одном иерархическом списочном методе. По самому кластеру, кроме его имени, версии и типа, никакой дополнительной информации не требуется.

Датацентр

В датацентре тоже нужно поддерживать обновление кластеров. Для этого в датацентре реализуется метод, принимающий на вход идентификаторы обновляемых кластеров, версию для обновления и метаданные по версии дистрибутива из хранилища дистрибутивов.

Метод датацентра по идентификатору кластера должен вычитать параметры кластера, сформировать на их основе helm-чарт и запустить его в Kubernetes. По итогам обновления кластера датацентр отчитывается Хоттабычу с помощью коллбэк-метода, аналогично обновлению облачных приложений.

Поддержка новых типов кластеров

Для того, чтобы научиться обновлять новые типы кластеров, требуется выполнить следующие действия:

1. В Kubernetes описать оператора и Custom Resources для того, чтобы сам Kubernetes знал, каким образом проводить обновление кластера.
2. В Datacenter указать в коде какое имя Custom Resources используется для этого типа кластера.
3. В Управлении облаком для нового типа кластеров добавить настройки helm-чарта и выставить для новых кластеров флаг "Развернут в Kubernetes".
4. Доработать сборку дистрибутива кластера: сборка должна выкладывать в хранилище дистрибутивов Хоттабыча json-файл со списком образов дистрибутива для того, чтобы в момент копирования дистрибутива между хранилищами вызывалось копирование образов контейнеров кластеров между разными image-store.

Скрипты

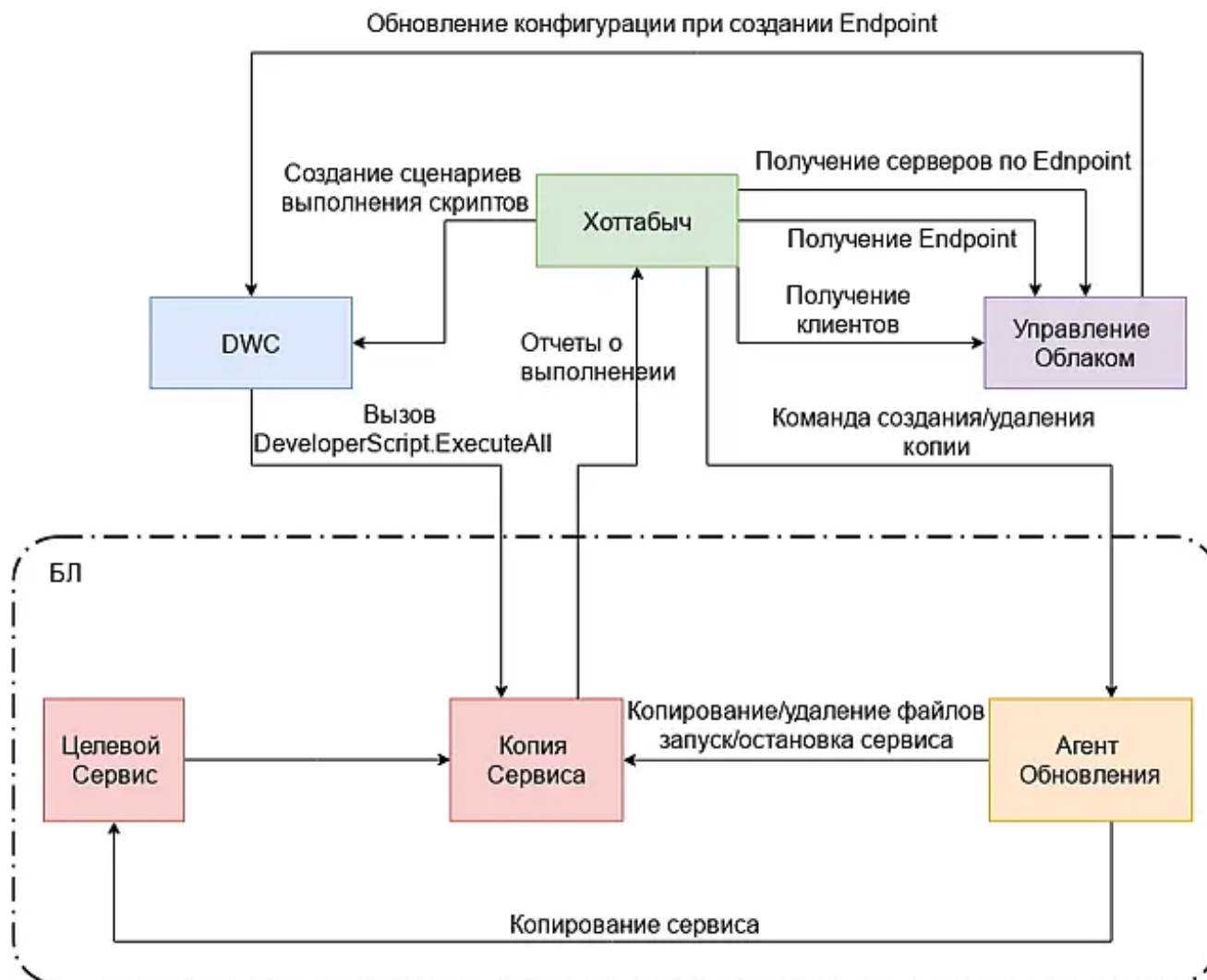


Рисунок 18. Структура процесса выполнения скриптов

Структура процесса выполнения скриптов основана на стандартном пофазном процессе обновления в Хоттабыче.

Фаза доставки

Осуществляется доставка утилиты обновления.

Фаза подготовки

Не выполняется.

Фаза обновления

Внедрение скриптов.

Утилита обновления находит расположение целевого сервиса и создает копию сервиса в отдельной папке.

Утилита обновления скачивает архив с модулем выполнения скрипта, архив с прикладным скриптом. Архивы распаковываются в папку с модулями бизнес-логики копии сервиса.

Название папки модуля выполнения скрипта - DeveloperScript_sbis-script-module, а для папки скрипта - "DeveloperScript_%ИмяАрхива%". В случае, если в архиве скрипта нет описания модуля s3mod, файл генерируется автоматически с добавлением зависимости от модуля Python Core, необходимым для импорта содержащихся в скрипте .py файлов. Затем копия сервиса запускается для того, чтобы внедренные модули были доступны для вызова.

Хоттабыч запрашивает клиентов сервиса из облака, и сохраняет их идентификаторы в таблицу "ClientDatabase". Если сервис не персональный, клиентом является сам сервис. В таблице присутствует связь с таблицей "ИсторияОперацийЛогики", содержащей описания всех логик участвующих в обновлении. Каждый клиент/сервис связан соответствующей логикой БД, для понимания какой клиент/сервис к какой БД принадлежит.

Генерация сценариев DWC

По полученным клиентам и сервисам Хоттабыч генерирует сценарии выполнения в DWC используя АПИ модуля "Workflow Coordinator Client-Py". По каждой базе данных сервиса создается один или несколько сценариев с последовательным выполнением задач. Количество сценариев зависит от настройки приложения (на котором запускается скрипт) "Число потоков конвертера для фаз исполнения скриптов".

Логический смысл настройки — максимальное число одновременно обрабатываемых клиентов в скрипте, так как во время выполнения одного сценария скрипт выполняется только на одном клиенте. Реальное число зависит от количества задач, которое могут одновременно обрабатываться сервисом, зависит от размера пула служебных задач, числа бизнес-логик и загрузки сервиса.

Каждый сценарий содержит два типа задач:

- `DeveloperScript.ExecuteAll` — задача для сервиса, исполняющего скрипт, выполняет скрипт разработчика. Возвращает результат задачи, содержащий информацию о результате выполнения скрипта, он передается в следующую задачу сценария.
- `ClientDatabase.ScriptDone` – задача для Хоттабыча, здесь происходит сохранение информации о результате выполнения скрипта, полученное из предыдущей задачи `DeveloperScript.ExecuteAll`.

Для каждого клиента/сервиса БД создается по одной задаче каждого типа.

Выполнение скрипта в методе `DeveloperScript.ExecuteAll`

`DeveloperScript.ExecuteAll` — метод, содержащийся в модуле выполнения скрипта.

Параметры метода:

UpdateID	Идентификатор обновления
ClientDatabaseID	Идентификатор записи в таблице "ClientDatabase"
VersionManagerService	Системное имя сервиса "Управление версиями"
Guid	Идентификатор запуска скрипта
Personal	Признак того, что сервис персональный
DoneMethods	Выполненные ранее методы скрипта, в рамках данного обновления.

Перед выполнением скрипта вызывается метод Хоттабыча "`ClientDatabase.CheckScript`" для проверки актуальности идентификатора запуска скрипта (параметр `Guid`). При старте выполнения определяются методы, содержащиеся в объекте `DeveloperScript`, среди них отбираются те, методы, что не содержат параметров и не содержатся в параметре `DoneMethods`.

Отобранные методы запускаются поочередно через `FutureInvoke`, но одновременно выполняется только один метод. В это время основной поток периодически отчитывается в Хоттабыч методом `ClientDatabase.ReportAlive` для того, чтобы следить за работой скрипта. Частота отчетов задается параметром облака "Управление версиями.Скрипты.ЧастотаОтчетов". После завершения всех методов `DeveloperScript` метод `DeveloperScript.ExecuteAll` возвращает запись с результатами выполнения, которая передается в Хоттабыч задачей `ClientDatabase.ScriptDone`.

Обработка ошибок

В случае возникновения ошибок в методах скрипта, они отловятся в блоке `try ... catch` в методе `DeveloperScript.ExecuteAll`. В случае ошибок в инфраструктуре, например, когда DWC не может сделать запрос на сервис, выполняющий скрипт, срабатывает обработчик ошибок сценариев DWC. При ошибке внутри скрипта в журнал выполнения публикуется сообщение о внутренней ошибке выполнения скрипта. При срабатывании обработчика ошибок сценариев DWC публикуется сообщение о внешней ошибке выполнения скрипта.

Хранение состояния выполнения

По каждому клиенту/сервису хранится состояние скрипта и дата его изменения в таблице `ClientDatabase`:

- Не выполнен
- Выполнен
- Ошибка
- Выполняется

Начальное состояние всех клиентов "Не выполнен". После вызова метода `ClientDatabase.ScriptDone` состояние меняется либо на "Выполнен", либо на "Ошибка". При получении вызова "`ClientDatabase.ReportAlive`" клиент переходит в состояние "Выполняется".

Публикация сообщений в журнал выполнения — дорогая операция, так как каждое сообщение приводит к перевычислению состояния обновления. Для этого результаты выполнения скрипта сохраняются в таблицы `ScriptMessage` (для хранения сообщений о выполнении или ошибке в скрипте) и `ScriptMethod` (для хранения информации о выполненных в скрипте методах). На таблицах отключен `write-ahead log` для быстрого действия. Задача планировщика `ClientDatabase.ProcessScriptResponses` служит для обработки записей в этих таблицах.

Обработка сообщений задач планировщика `ClientDatabase.ProcessScriptResponses`

Задача запускается каждую минуту (если в данный момент работает другой экземпляр задачи — то не запускается).

Выбирается пачка записей из таблицы "ScriptMessage" (размер задается параметром "Управление версиями.Скрипты.РазмерПачкиСообщений"). По отобранным сообщениям вычисляются статусы логик баз данных с клиентами. Например, если обработаны все клиенты, то логика помечается завершенной. Если были ошибки, то ошибочной. Генерируются сообщения для журнала обновления. Отобранные записи из таблицы "ScriptMessage" удаляются. Цикл повторяется пока есть сообщения в таблице "ScriptMessage".

Также эта задача следит за зависанием скриптов. Так как по каждому клиенту хранится дата последней смены статуса, можно определить, что скрипт перестал выполняться. В таком случае публикуется сообщение с ошибкой, о том, что выполнение скриптов прервано по таймауту. Длительность таймаута задается параметром облака "Управление версиями.Скрипты.ОжиданиеОтчета".

Прерывание выполнения скрипта

При прерывании скрипта Хоттабыч отменяет сценарии DWC. Отмена сценария в DWC приводит к отмене еще не сделанных асинхронных вызовов.

Для полной остановки работы методов скриптов при прерывании происходит остановка копии сервиса. Перед остановкой проверяется наличие асинхронной очереди по скриптовой БЛ в облаке. В случае наличия очереди до истечения определенного таймаута, прерывание не считается успешным. В данной ситуации возможно только возобновить прерывание.

Фаза подтверждения

Основной задачей подтверждения на узлах скриптовых является:

1. отсутствие асинхронных очередей по этим узлам
2. отсутствие незавершенных процессов скриптовых БЛ.

Для выполнения этих требований подтверждение выполняется в следующем порядке:

1. перезапускается узел скриптовой БЛ.
2. проводится анализ очередей двумя алгоритмами: через облако и по анализу текущей очереди запросом ?load к узлу.
3. завершается работа процесса скриптовой БЛ.

В таблицу ScriptHistory сохраняется информация о выполнении скрипта. Если скрипт на персональном сервисе был выполнен не до конца, в эту таблицу сохраняются уже обработанные клиенты, это позволяет доделать скрипт позднее, после завершения обновления.

Разворот пакетов данных

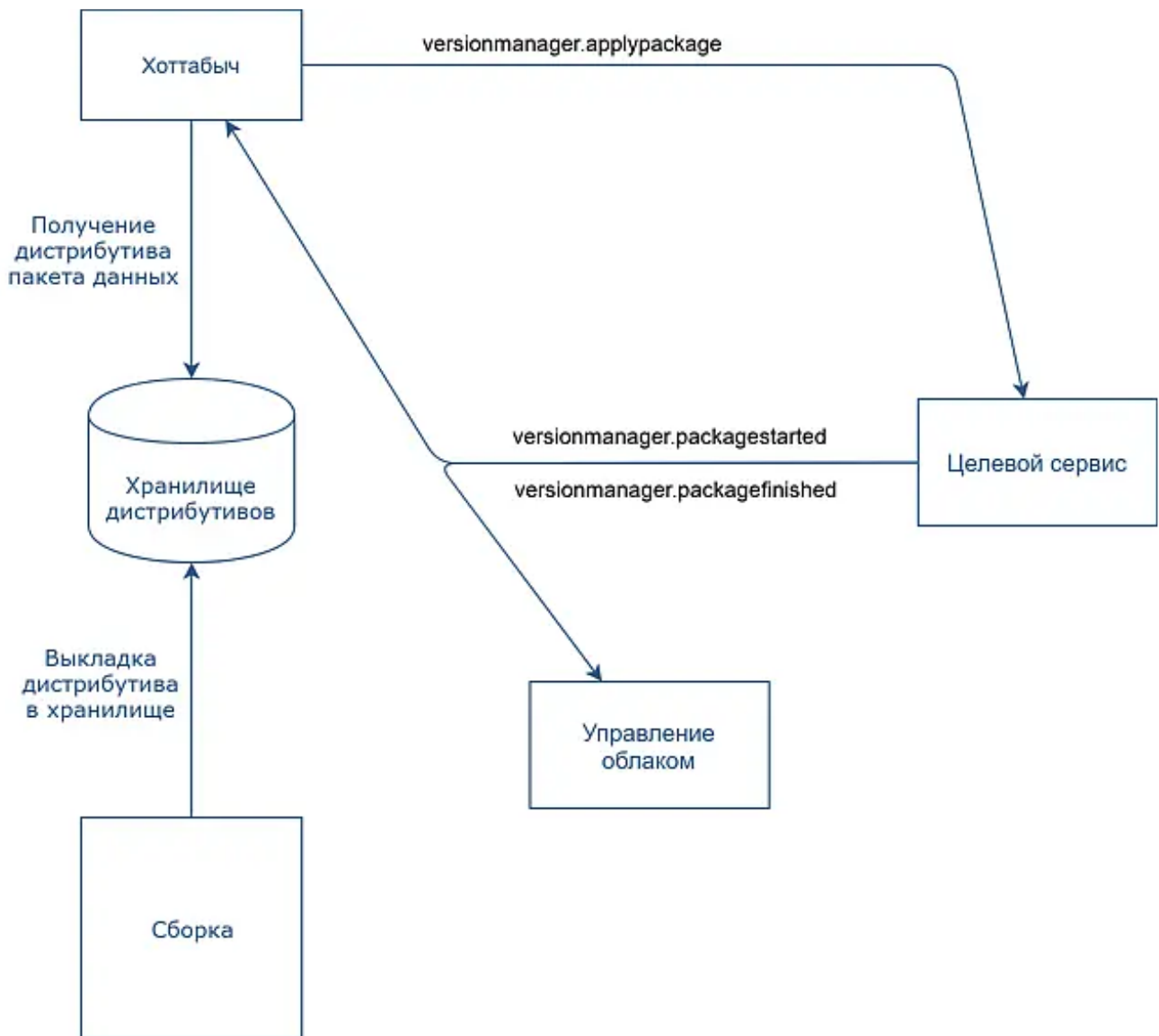


Рисунок 19. Архитектура процесса разворота данных

1. Организуется сборка, которая собирает дистрибутив пакета данных, требуемых сервису. Дистрибутив выкладывается в ХД.
2. В интерфейсе Хоттабыча у приложения указывается дистрибутив пакета данных.
3. Обновляющий в Хоттабыче запускает процесс разворота собранного пакета данных на целевом сервисе.
4. Хоттабыч публикует серверное событие, содержащее информацию о пакете, и ссылку на дистрибутив пакета.
5. Сервис принимает серверное событие, скачивает дистрибутив по ссылке, выполняет внутреннюю логику по развороту данных.
6. По завершению разворота сервис публикует серверное событие об окончании разворота данных, которое ловит Хоттабыч и завершает процесс.

Сборка

Организуется сборка, которая собирает дистрибутив пакета данных, требуемых сервису. Дистрибутив выкладывается в ХД.

Хоттабыч

1. В интерфейсе Хоттабыча у приложения указывается дистрибутив пакета данных.
2. Обновляющий в Хоттабыче запускает процесс разворота собранного пакета данных на целевом сервисе.
3. Хоттабыч публикует серверное событие **versionmanager.applypackage**, содержащее информацию о пакете, и ссылку на дистрибутив пакета.
4. Хоттабыч ждет события **versionmanager.packagestarted**, сигнализирующее, что разворот пакета начался.

Хоттабыч ждет события **versionmanager.packagefinished**, сигнализирующее, что разворот пакета завершился.

Целевой сервис

1. Реагирует на событие `versionmanager.applypackage`, получается данные о пакете.
2. Запускает метод, обрабатывающий данные и посылает событие `versionmanager.packagestarted`
3. По завершению разворота или при возникновении ошибки сервис посылает событие `versionmanager.packagefinished`

Управление облаком

Сервис управления облаком подписывается на событие `versionmanager.packagefinished` и обновляет текущую версию пакета сервиса в облаке.

Агент обновления

Агент исполняет команды приходящие от БЛ Хоттабыча. Сам по-себе он не реализует большого числа функций, только минимально необходимые для выполнения команд. Команда — это запрос на запуск утилиты обновлений `sbis-update-exes` с заданными аргументами. Перед исполнением команды, по необходимости, скачивается и распаковывается архив с этой утилитой. Внутри `sbis-update-exes` совершает необходимые действия для команды. О статусе выполнения БЛ Хоттабыча узнаёт по сообщениям от агента. В сообщениях пишется `stdout` от `sbis-update-exes` и служебные пометки от агента (о завершении команды, ошибки и т.п.).

Все функции реализованные в утилите могут быть реализованы напрямую в агенте. Но они сделаны в утилите, поскольку отладка, обновление и разработка непосредственно самого агента технически сложнее, чем отдельного исполняемого файла. И так при добавлении новых функций довольно редко требуются правки в сам агент, достаточно просто указать ссылку на утилиту необходимой версии.

Перед выполнением утилиты `sbis-update-exes`, необходимые исполняемые файлы доставляются на сервер с агентом посредством вызова `UpdateAgent.Download` с указанием нужной ссылки.

Для выполнения утилиты `sbis-update-exes` БЛ Хоттабыча вызывает метод агента `UpdateAgent.ExecuteAsync`. По результату выполнения этого метода запускается процесс `sbis-update-exes` с требуемыми аргументами. Для контроля этого процесса метод `UpdateAgent.ExecuteAsync` порождает несколько потоков (рис. 20):

- `ExecuteCommandThread` запускается (`thread::detach`) непосредственно из метода `UpdateAgent.ExecuteAsync` и работает в фоне после завершения метода. Из этого потока запускается процесс утилиты `sbis-update-exes` и происходит ожидание его завершения. Для взаимодействия с процессом утилиты отделяются два других потока (`SendReportThread` и `WaitInterruptionThread`). По завершению работы утилиты они прерываются и сливаются (`thread::join`) обратно в `ExecuteCommandThread`.
- `SendReportThread` читает `stdout` процесса утилиты и записывает его в синглтон `Messages`, который накапливает записи в `std::map`, где в качестве ключа используется адрес хоста Хоттабыча от которого пришёл вызов `UpdateAgent.ExecuteAsync`.
- `WaitInterruptionThread` раз в секунду опрашивает синглтон `InterruptionObject` на предмет того, что не было ли запроса на прерывание выполнения утилиты. Если такой запрос был, то процессу утилиты отправляется сообщение о завершении.

Метод `UpdateAgent.Interrupt` принимает запросы на прерывание процесса утилиты. Запуски утилит идентифицируются по адресу хоста Хоттабыча, ключу из таблицы ИсторияОперацийЛогике и фазе. По результату выполнения `UpdateAgent.Interrupt` в синглтоне `InterruptionObject` выставляется флаг о запросе прерывание соответствующего процесса.

Поток `SendReportsThread` (не путать с `SendReportThread`) раз в 5 секунд агрегирует накопленные в синглтоне `Messages` сообщения и рассылает нужным БЛ Хоттабыча. Это происходит посредством вызова метода Хоттабыча - `Отчеты.Агента`.

Метод `UpdateAgent.Upgrade` обрабатывает запросы на обновление самого агента. Сначала через синглтон `InterruptionObject` проверяется, что текущий момент нету никаких запущенных процессов утилиты, иначе обновление агента не стартует. Для обновления агента запускается утилита с переданными в `UpdateAgent.Upgrade` параметрами. Процесс утилиты отделяется от агента таким образом, чтобы он не был дочерним процессом агента — это нужно для того, чтобы утилита могла перезапустить агента.

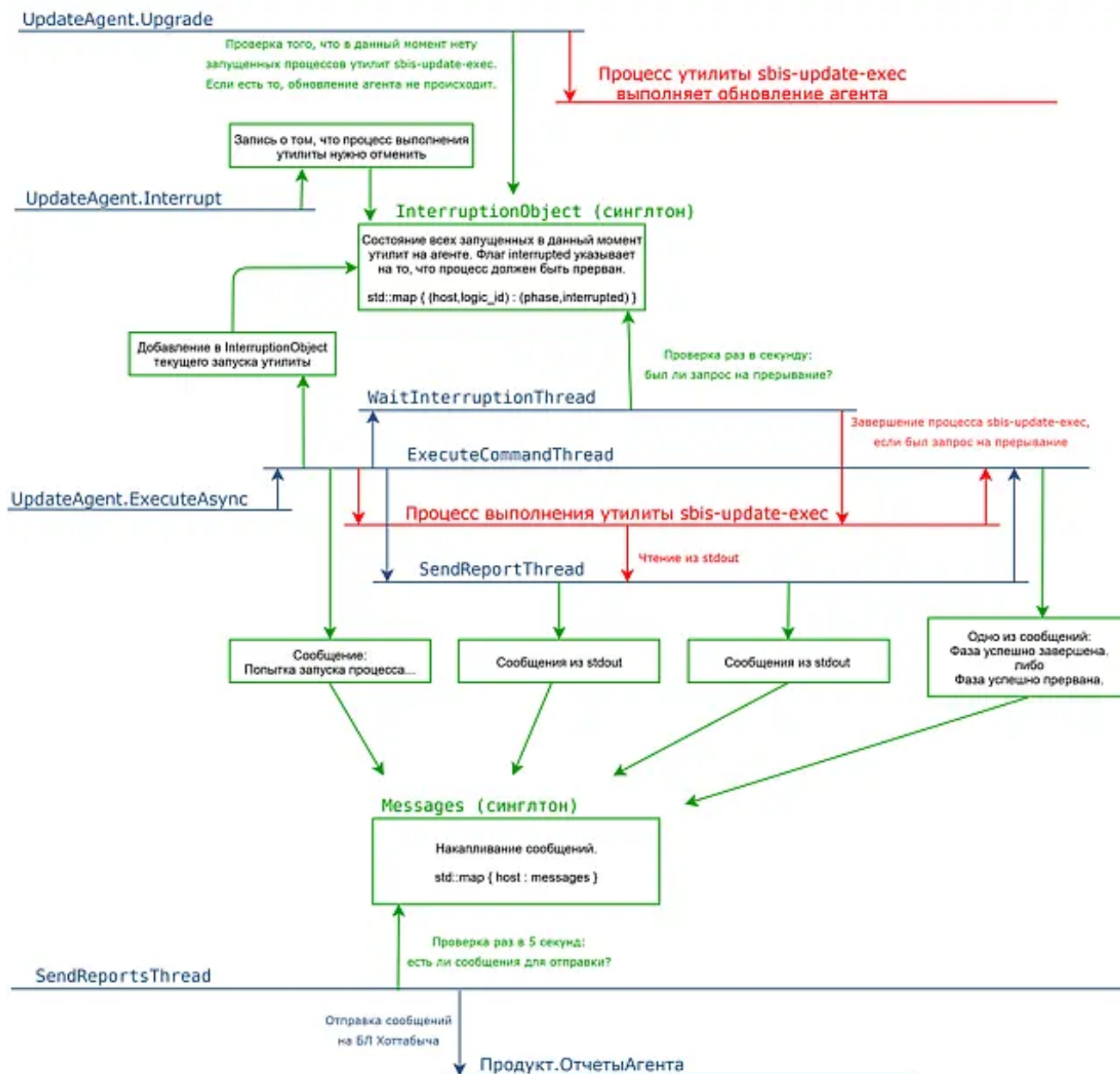


Рисунок 20. Поток, порождаемый методом UpdateAgent.ExecuteAsync

Остальные методы агента:

- `UpdateAgent.ExecuteSync` — синхронно выполняет утилиту с переданными параметрами, и по результату возвращает строку с `stdout` выполненного процесса.
- `UpdateAgent.OperatingSystem` — запрос о том под управлением какой операционной системы работает данный сервер.
- `UpdateAgent.ServerInfo` — запрос общей информации о данном сервере (объем диска, оперативной памяти, число ядер и т.д.)
- `UpdateAgent.Version` — запрос версии агента.

Обновление агентов из Управления облаком

Метод `VersionManager.CommandToAgent`.

`VersionManager.CommandToAgent(Agents: Json, Operation: Json): Record` - метод вызова операции на агентах.

Вызывается из Управления облаком при выполнении операции обновления агентов. Общее описание метода:

- **Agents: Json** - список агентов, над которыми производится операция. Формат:

```

1 {
2   Host1: [ { Port: String, Directory: String, SystemName: String,
3             ServerPool: Int32, AgentPort: String },
4             { Port: String, Directory: String, SystemName: String
5             } ]
6   Host2: ...

```

```
5 | }
```

- **Operation: Json** - описание операции. Формат:

```
1 | {
2 |     <название операции>: String - bl_reboot, bl_stop, bl_start,
   |     bl_invoke, agent_update.
3 |     {
4 |         update_id: Int32 - идентификатор обновления, не обязательно;
5 |         id: String - идентификатор операции, должен уникально
   |         идентифицировать запрос операции;
6 |         modifier: String - модификатор операции (synchronously,
   |         sequentially) синхронно или последовательно. При обновлении агентов
   |         не используется, операция всегда выполняется последовательно;
7 |         Object: String - для bl_invoke, название объекта БЛ;
8 |         Method: String - для bl_invoke, название метода БЛ;
9 |         Arguments: Record - для bl_invoke, аргументы метода.
10 |     }
11 | }
```

- **Result: Record**

```
1 | {
2 |     Agents: Json - Тот же самый json что на вход, только с
   |     добавленными @AgentOperationsServer (идентификаторы записей из
   |     таблицы списка серверов, входящих в операцию на агентах);
3 |     Operation: Json - Тот же самый json что на вход, только с
   |     добавленным @AgentOperationsHistory (идентификатор записи в таблице
   |     задач, выполняемых на агентах обновления).
4 | }
```

Общая логика процесса обновления агентов по команде из Управления облаком:

1. Создается запись в таблице AgentOperationsHistory и связанные записи в таблице AgentOperationsServer. Идентификаторы записей в дальнейшем используются при публикации событий.
2. На всех обновляемых серверах параллельно запускается операция загрузки утилиты.
3. Последовательно запускаются операции обновления агентов.

В ходе выполнения операций публикуются события versionmanager.agent_operation_changed, versionmanager.agent_operation_server_changed, versionmanager.agent_operation_journal_changed.

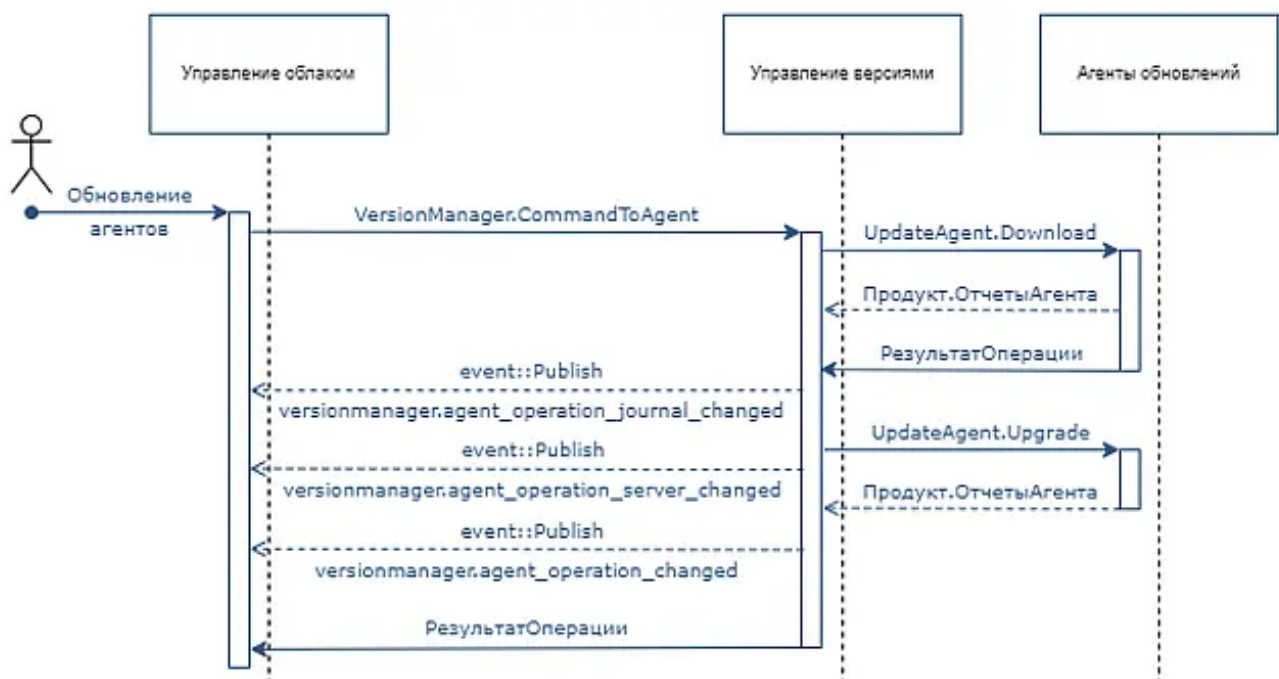


Рисунок 21. Общая диаграмма процесса обновления агентов из Управления облаком

Описание события versionmanager.agent_operation_changed:

- Visibility = evSERVER_ONLY;
- PublicationPolicy = evIMMEDIATELY;
- Delivery = evGLOBAL;
- message - сериализованная HashTable:
 - @AgentOperationsHistory - идентификатор записи из соответствующей таблицы;
 - Identifier - идентификатор из параметра Operation:id метода VersionManager.CommandToAgent;
 - Status - статус операции;
 - Error - признак ошибки.

Описание события versionmanager.agent_operation_server_changed:

- Visibility = evSERVER_ONLY;
- PublicationPolicy = evIMMEDIATELY;
- Delivery = evGLOBAL;
- message - сериализованная HashTable:
 - @AgentOperationsHistory - идентификатор записи из соответствующей таблицы;
 - @AgentOperationsServer - идентификатор записи из соответствующей таблицы;
 - Identifier - идентификатор из параметра Operation:id метода VersionManager.CommandToAgent;
 - LogicIdentifier - из параметра Agents:LogicIdentifier ? метода VersionManager.CommandToAgent;
 - Host - хост агента из параметра Agents:Host метода VersionManager.CommandToAgent;
 - Port - порт агента из параметра Agents:Port метода VersionManager.CommandToAgent;
 - Directory - виртуальный каталог из параметра Agents:Directory метода VersionManager.CommandToAgent;
 - DaemonName - системное имя сервиса из параметра Agents:SystemName метода VersionManager.CommandToAgent;
 - Status - статус операции;
 - Error - признак ошибки;
 - Message - Текстовое сообщение.

Описание события versionmanager.agent_operation_journal_changed:

Полностью соответствует формату versionmanager.agent_operation_server_changed.

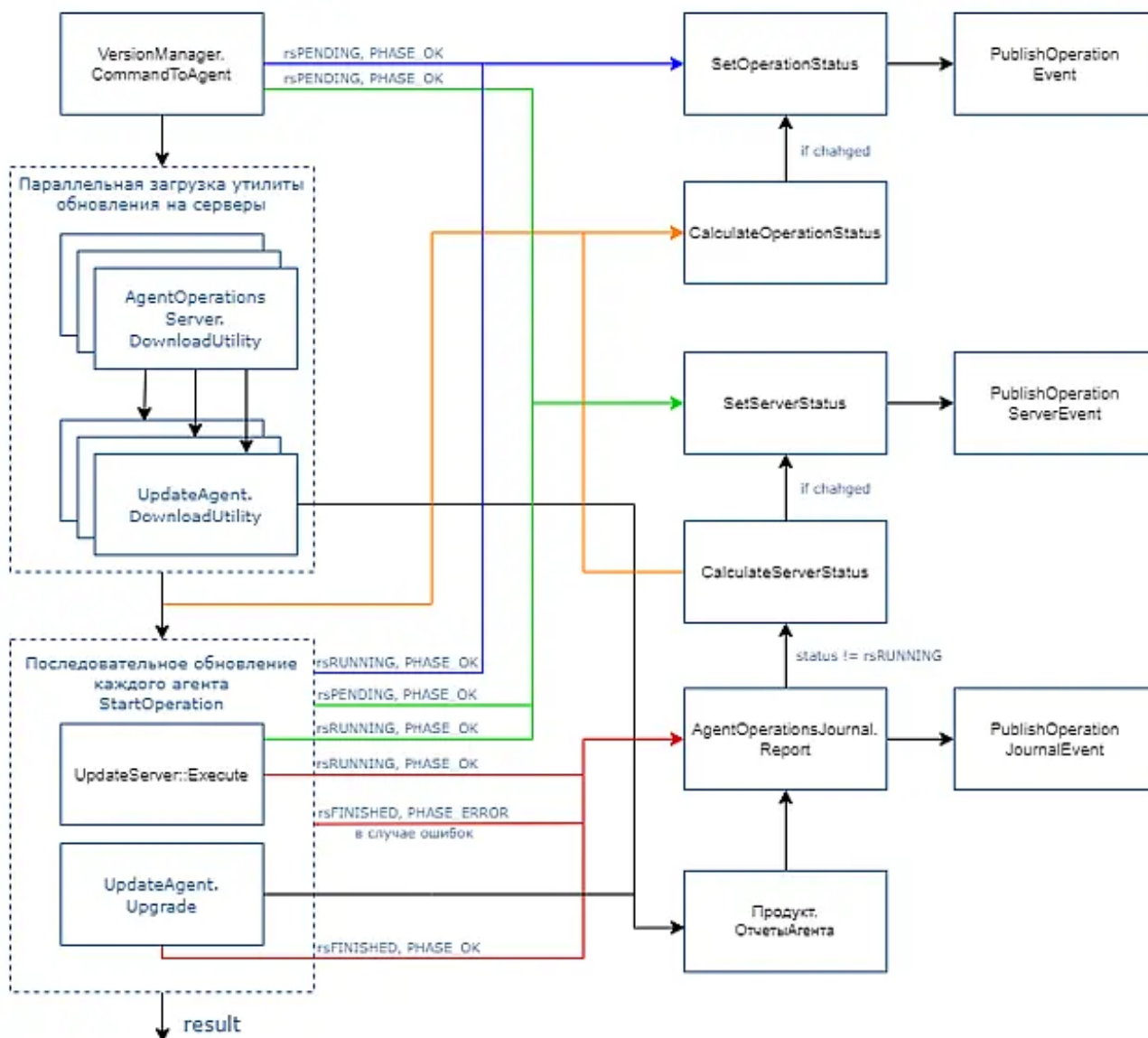


Рисунок 22. Схема публикации событий при обновлении агентов

Утилита обновления

Общее управление и координация всех процессов Хоттабыча происходит через его БЛ. При необходимости выполнения некоторого действия на конкретном сервере посылается команда (гpc-запрос) установленному на нём агенту. Агент скачивает дистрибутив утилиты из ХД соответственно версии БЛ Хоттабыча. Команда представляет собой путь до утилиты и аргументы её запуска. При вызове утилита разбирает переданные аргументы и вызывает реализованную в ней нужную C++ функцию. Примеры таких функций: запуск фазы доставки обновления или перезагрузка сервера.

Аргументы утилита получает в качестве ссылки на файл в сервисе file-storage или по id в file-transfer. БЛ Хоттабыча формирует и аплоадит этот файл на file-storage или file-transfer. Для выполнения команды могут требоваться и другие файлы (например, дистрибутив jinnee и его конфигурацией), они передаются таким же образом. Что использовать file-storage или file-transfer определяется параметром конфигурации Хоттабыча, спецификой операции / обновляемым продуктом.

Работа с аргументами утилиты представляет определённые трудности: они передаются парами строковый ключ – значение. Вариантов использования утилиты множество и проследить где, какие значения нужны бывает достаточно сложно, кроме того между разными вариантами есть общая логика работы. Как сделать лучше – открытый вопрос. На данный момент есть идея (в процессе реализации) разбить аргументы на группы, с каждой такой группой связана некоторая логика (например, набор параметров нужный для аутентификации в облаке). В коде каждая группа – это C++ тип (struct с полями). Таким образом, за счёт статической типизации, получается легко проследить, как используются аргументы – в нужных местах явно фигурирует один и тот же тип, и компилятор способен выдавать ошибки при несоответствии (код находится в общих для утилиты и БЛ *.h файлах). Для сериализации C++ struct написан макрос STATIC_RECORD (в реализации используется класс Record из платформы).

Вспомогательные классы для обновления сервисов

В библиотеке sbis-update-generic реализованы два класса, используемые в том числе и в функционале перемещения файлов сервиса:

1. `AbstractSvcPartsManager`. Класс инициализируется путем до корня сервиса и двумя списками: игнорируемых файлов и папок. Класс предназначен для обхода файлов сервиса согласно разбивке сервиса по частям. Во время обхода для каждой части вызываются два коллбэка: один — для разрешения, нужно ли вообще обходить эту часть, второй — действие, которое с этой частью необходимо выполнить.
2. `ServiceHash`. Класс инициализируется теми же аргументами. Предназначен для вычисления хэша сервиса для проверки корректности обновления. В случае несовпадения общего хэша сервиса, класс выдает хэши всех частей сервиса для возможности анализа, какие именно части не совпали по хэшу.

На основе `AbstractSvcPartsManager` в `sbis-update-exec` был создан класс `ServiceFilesMover`, инициализируемый путями откуда перемещать и куда, а также опциональным параметром — списком частей сервиса. Данный класс используется для перемещения файлов сервиса любого типа дистрибутива, даже старого модульного. Если список частей в класс не передавался — перемещение производится для всех файлов сервиса. Класс наследуется от `AbstractSvcPartsManager` и реализует в себе логику работы его коллбэков.

В коллбэке действий в `ServiceFilesMover` используется новый метод "быстрого" перемещения файлов `FastMove`, реализованная на основе методов `std::filesystem`. Логика его работы следующая:

1. Если исходный путь не существует, значит и перемещать нечего — выходим с положительным результатом.
2. Если путь назначения существует — значит удаляем его с помощью функции `std::filesystem::remove_all`.
3. Пытаемся сделать переименование исходного пути в путь назначения с помощью `std::filesystem::rename`. Ее преимущество по сравнению с бустовой функцией в том, что переименование происходит даже при наличии вложенных путей без необходимости ручного обхода всего дерева.
4. `rename` может не пройти, если файлы находятся на разных логических разделах. Тогда выполняется сначала копирование данных функцией `std::filesystem::copy` с опцией `recursive`, а затем удаление исходных данных функцией из п.2.

Алгоритм выполнения обновления сервисов БЛ

Утилита обновления от Хоттабыча на фазе подготовки получает список частей сервиса для загрузки с их ссылками из хранилища дистрибутивов. Утилита скачивает части в папку подготовки и фиксирует в специальной папке факт успешного скачивания части, чтобы при возобновлении упавшей подготовки не перекачивать часть повторно. Части скачиваются и распаковываются в раздел `/opt/sbis`, поэтому при обновлении на такой тип дистрибутива не расходуется место в корневом разделе серверов и не тратится время и ресурсы на копирование данных между разными логическими дисками. После распаковки всех частей новые файлы в папке подготовки уже лежат в нужной структуре, в которой они будут находиться в рабочем сервисе.

На фазе обновления при обновлении дистрибутива по частям возможны два варианта передачи аргументов в утилиту:

1. передача признака обновления всех частей сервиса;
2. передача массива частей обновляемых сервисов.

При обновлении всех частей сервиса, перемещение происходит практически так же, как раньше: сначала все файлы сервиса перемещаются в бэкап, а затем все файлы из папки подготовки перемещаются в папку с сервисом. В этом случае в `ServiceFilesMover` не передается аргумент со списком частей сервиса.

При передаче массива частей утилите приходят относительные пути до обновляемых частей сервиса. Этот массив передается в `ServiceFilesMover`. Дальше логика работы следующая:

1. В бэкап из обновляемого сервиса перемещаются только те части, которые были переданы в `ServiceFilesMover`. Это список устаревших частей, которые больше не потребуются обновляемому сервису. В результате перемещения в бэкап, в сервисе останутся только те части, которые в результате обновления остались неизмененными.
2. Из подготавливаемой папки тем же `ServiceFilesMover` перемещаются все новые части в папку с сервисом.

В бэкапе при этом у нас остались устаревшие части сервиса. Поэтому в случае отката обновления, утилите передается массив новых частей, чтобы их сначала "изъять" из сервиса, а затем устаревшие части из бэкапа возвращаются в сервис.

В случае перемещения файлов сервиса в новое местоположение, после выполнения вышеуказанных инструкций выполняется перемещение неизменившихся данных с помощью `ServiceFilesMover` из старого местоположения в новое.

После выполнения перемещения файлов производится проверка `md5` сервиса классом `ServiceHash`, который инициализируется путем до обновленного сервиса. Вычисленный хэш сравнивается с тем хэшем, который был рассчитан на этапе сборки дистрибутива утилитой `sbis-distribution-preparation`. Если сравниваемые хэши не совпадают, выдается ошибка несовпадения хэша, после чего происходит анализ расхождения хэшей всех частей: тех, что были в исходном дистрибутиве и частей, находящихся в сервисе. Сравнение хэшей частей происходит следующим образом: имеются два множества частей сервиса, со стороны исходного дистрибутива и со стороны обновленного сервиса; эти множества сравниваются, исключая из них части с одинаковым хэшем. Оставшиеся части в каждом множестве — части, у которых не совпали хэши. Их список выводится в ошибку.

Алгоритм работы утилиты при конвертации БД

При конвертации БД утилита и джинн работают циклами прохода по схемам баз. На каждом цикле прохода джинн спрашивает утилиту, можно ли ей конвертировать обрабатываемую схему, а утилита отвечает в зависимости от типа и настроек обновления.

Для обновлений с типами "легкая конвертация", "полное обновление", "полное последовательное", "конвертация мастер-реплика" это sccPROCEED, что означает "можно выполнять конвертацию схемы".

Для обновления с типом "полное обновление без остановки работы (посхемное)" ответ утилиты зависит от нескольких факторов:

1. заданы ли списки конвертируемых и игнорируемых клиентов;
2. удалось ли заблокировать клиента для обновления;
3. пришла ли необходимая информация по клиенту из облака.

Возможные значения:

- sccPROCEED - можно выполнять конвертацию схемы;
- sccPERMANENTLY_SKIP - схему нужно пропустить, считая сконвертированной. Схема была исключена из обновления в соответствии со списками конвертируемых и игнорируемых клиентов. Для списка конвертируемых схем исключаются все, что не входят в этот список; для списка игнорируемых схем — наоборот, исключаются все, что в него входят. Пропущенные схемы сохраняются в отдельный контейнер и, при успешном завершении фазы, с помощью метода PerSchemaParams.AppendSkippedSchemas передаются в Хоттабыч для включения их в dbschema для следующей фазы;
- sccTEMPORARY_SKIP - отложить конвертацию схемы, т.к. еще не пришла необходимая информация по клиенту из облака;
- sccMARK_AS_ERROR - схему нужно пропустить и включить в список несконвертированных, т.к. не удалось заблокировать клиента для обновления.

При посхемном обновлении после каждого цикла прохода схем утилита запрашивает новую пачку клиентов из облака, размером пачки управляет утилита. Также в этом утилита через метод

ИсторияОперацийПродукта.PerschemaParams запрашивает у Хоттабыча параметры обновления, включающие списки конвертируемых и игнорируемых клиентов, число потоков конвертера, таймаут конвертации схемы, размер пачки уведомляемых клиентов. Изменившиеся параметры обновляются в утилите, а новое значение числа потоков конвертера передается в колбэк static ConvertationControlCommand SetThreadCount(size_t value).

Если параметр "таймаут конвертации схемы" имеет ненулевое значение, утилита производит замер времени конвертации схемы и, если после начала конвертации схемы прошло больше времени, чем задано в параметре, кидает предупреждение в журнал логики базы данных о превышении таймаута конвертации схемы.

Утилита обновления при запуске основной фазы посхемного обновления заводит контейнер, хранящий идентификаторы проблемных клиентов обрабатываемой базы. При получении колбэка от Genie об ошибке конвертации схемы, соответствующий клиент заносится в контейнер, а в Хоттабыч через отчеты агента с тегом "FAILED_CLIENTS" отправляется сообщение об общем количестве проблемных клиентов по базе, в которой произошла ошибка. При последующей успешной конвертации клиента, входящего в этот контейнер, клиент из него извлекается, а в облако отсылается актуализированное общее число проблемных клиентов. На каждом цикле прохода схем утилита скидывает данные, сохраненные в контейнере на диск, на случай непредвиденного завершения процесса утилиты. Данные также скидываются при завершении фазы конвертации с ошибкой. При успешном завершении работы джинна контейнер и сохраненные очищаются.

Логирование утилиты

По умолчанию в файловые логи утилиты пишутся только ключевые действия утилиты. Но параметром облака Управление версиями.Обновления.Логи.ПодробноеЛогированиеУтилиты можно включить расширенное логирование утилиты, при котором во многих функциях включено логирование вызова этих функций с их аргументами. Параметр из облака передается утилите через UtilityConfig. Этот параметр стоит включать с осторожностью, так как ранее, когда он был включен по умолчанию, на каждое обновление утилита писала логов примерно 500Мб, создавая тем самым дополнительную паразитную нагрузку на систему ХД.

Инструкция по отладке

Для запуска обновлений с локально собранной утилитой необходимо загрузить архив с ней (собирается vsrc/utills/utility_nix.x86_64/sbis-update-exec_nix.x86_64.zip) в репозиторий (Репозитории → admin.sbis.ru → нужная версия → Загрузить файл). Версия БЛ должна соответствовать (при необходимости можно изменить в version.xml в директории сервиса). На версию можно поставить замочек, чтобы очистка хранилища её не трогала.

Для отладки в дебаре локально можно запускать через QtCreator (Debug → Start Debugging → Start and debug external application), указав при этом аргументы из журнала сообщений Хоттабыча (см. рисунок 1). Исполняемый файл указать из bin в сборочной директории (так при пересборке будет указан актуальный файл). Необходимые библиотеки можно взять из архива с утилитой, скопировав их в директорию с исполняем файлом, либо настроив переменную окружения LD_LIBRARY_PATH для linux или PATH для windows.

Анализ ревизий модулей и работа с версиями модулей при обновлении сервиса для модульных дистрибутивов (устаревшее поведение, скоро будет убрано)

Утилита обновления на фазе подготовки по каждому разворачиваемому сервису получает файл modules.json, выглядящий примерно так:

```
1 {"bl_modules": {
```



```

2      "AMQP Client":{
3          "link":"http://fix-distr-
storage.sbis.ru/api/v1/distribution_files/f64a89fe-cd6b-4e77-80a7-
1df540d8cf76",
4          "revision":"018b3cac893fb9bec887cd7d6e18535d",
5          "version":"19.610-853"},
6      "BL API Metadata Base":{
7          "link":"http://fix-distr-
storage.sbis.ru/api/v1/distribution_files/baf69b6b-f884-47d1-8c25-
02d1afa5f977",
8          "revision":"f961fb210c4b1324f57d628f39d515e7",
9          "version":"19.610-858"}},
10     "ui_modules":{
11         "Application":{
12             "link":"http://fix-distr-
storage.sbis.ru/api/v1/distribution_files/a5702209-4bf5-4862-819f-
878f0fd1f3fc",
13             "revision":"40d25a5e1c618409a7730eac881a361d",
14             "version":"99.00-2419"},
15         "Browser":{
16             "link":"http://fix-distr-
storage.sbis.ru/api/v1/distribution_files/fb0e01ae-6903-45a5-a747-
57d3071e5608",
17             "revision":"9d2bdabcd698b7e34e51f8cc555e8862",
18             "version":"99.00-2419"}}}

```

Файл содержит список всех модулей бизнес-логики и интерфейса сервиса с информацией: ссылка на архив с модулем в ХД, ревизия модуля, версия модуля. Все эти данные по модулям выдает Хоттабычу ХД по имеющимся в нем данным. При деплое сервиса утилита `jinnnee_lite` подкладывает в корень каждого модуля файл `file.checksum`, в содержимом которого содержится ревизия модуля.

Алгоритм разворота сервиса следующий:

1. Утилита скачивает из ХД `db-` и `core-` архивы сервиса, которые безусловно обновляются всегда. Распаковывает ее в отдельную папку, в которой формируется структура дистрибутива.
2. Проходим по файлу `modules.json` и проводим анализ установленных модулей сервиса на предмет расхождения ревизий. По итогам анализа формируем два списка модулей: неизменившиеся модули и модули с отличающейся ревизией.
3. Неизменившиеся модули копируем из существующего сервиса в папку подготовки дистрибутива.
4. Измененные модули перекачиваем из ХД и распаковываем в папку подготовки.
5. Проводим анализ зависимостей измененных модулей: для всех модулей, которые зависят от измененного модуля, версия модуля поднимается до версии измененного модуля в файле `modules.json`.
6. Запускается утилита `jinnnee_lite`, на вход которой подается путь до подготовленной папки-дистрибутива с модулями и файл `modules.json`. В процессе деплоя сервиса `jinnnee` проставляет в разворачиваемых модулях версии, указанные в `modules.json`.

Дальнейшее обновление сервиса происходит в штатном режиме, подменой файлов сервиса.

Серверы конвертации

Распределение конвертеров

Список активных серверов конвертации Хоттабыч получает из соответствующего кластера в Управлении облаком. После запуска обновления происходит назначение конвертеров на hosts БД обновляемого приложения. При назначении очередного конвертера происходит выбор наименее загруженного из них, о том как определяется загруженность рассказано далее.

У каждой выполняемой на конвертере задачи есть сложность. За сложность задачи условно принято количество потоков, которые `jinnnee` задействует при выполнении данной задачи. После назначения очередного конвертера для хоста БД определяется сложность задачи для основной фазы конвертации. Данная информация сохраняется в базу. Суммарная сложность планируемых задач для основной фазы как раз и определяет загруженность конвертера.

Ограничение одновременно выполняющихся задач

Перед отправкой очередной задачи на конвертер происходит подсчет веса уже запущенных на нём задач, проверка состояния системных ресурсов (la и free ram) и загрузки процессора (la) хоста ЦОДа, на котором расположен конвертер. Данные о системных ресурсах запрашиваются у агента на конвертере, загрузка процессора хоста — из мониторинга. В случае превышения установленных порогов, задача переходит в режим ожидания. После чего, примерно раз в минуту, происходит проверка возможности запуска задачи. Если время ожидания превышает заданный таймаут, задача завершается с ошибкой.

Схема базы данных

Схема сущностей

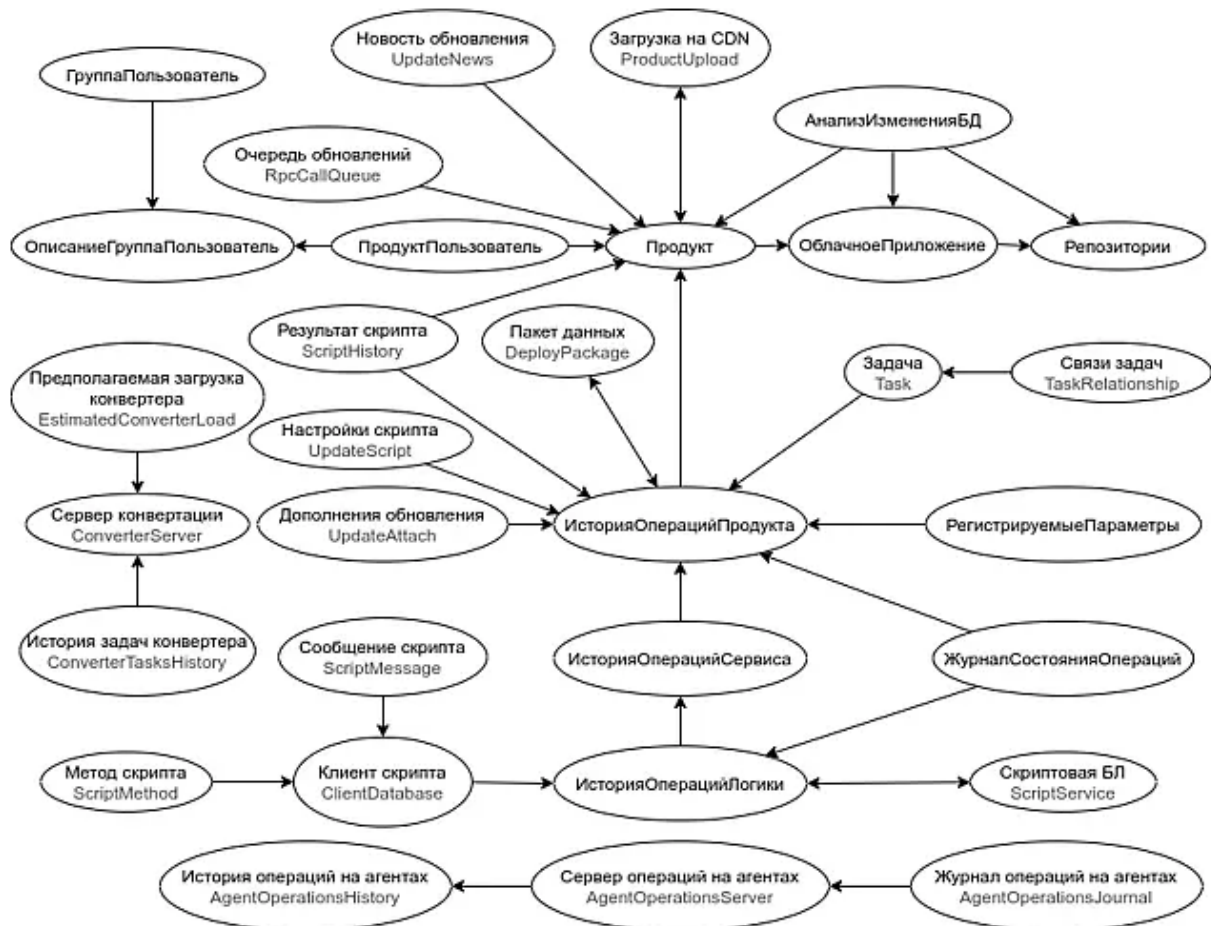


Рисунок 1. Схема сущностей управления версиями

Облачное приложение - используется для хранения записей настроек облака.

Новость обновления - таблица записей новостей. Новость показывается во время обновления приложения.

ГруппаПользователь - таблица связи между ОписаниеГруппаПользователь и пользователями облака.

ОписаниеГруппаПользователь — группы пользователей.

ПродуктПользователь - связи пользователей и групп пользователей с продуктами.

РегистрируемыеПараметры - список параметров для регистрации в облаке.

Клиент скрипта - хранит список клиентов для скриптов, состояние выполнения, ссылку на логику БД, к которой относится клиент, используется при выполнении скриптов.

Настройки скрипта - хранит настройки запуска одного скрипта.

Сообщение скрипта - таблица-очередь, хранит ответы от клиентов скрипта, по мере обработки – удаляются.

Метод скрипта - содержит информацию о выполненных методах скриптов, в конкретном выполнении скрипта.

Скриптовая БЛ - хранит настройки скриптовой БЛ для выполнения скрипта.

Результат скрипта - хранит результат выполнения скрипта.

Дополнения обновления - скрипты и патчи, относящиеся к конкретному обновлению.

АнализИзмененияБД - хранит базовую информацию об отчете анализа базы данных.

Репозитории - хранит информацию о подключенных репозиториях (сервис distribution storage).

Пакет данных - таблица для хранения данных обновления типа разворот пакета данных, расширяет таблицу ИсторияОперацийПродукта.

Задача - хранит задачи, выполняемые в процессе обновления и их параметры. Используется в асинхронном режиме обновления.

Связи задач - хранит зависимости между задачами. Используется в асинхронном режиме обновления для определения последовательности выполнения задач.

История операций на агентах - содержит записи задач, выполняемых на агентах обновления, такие как перезагрузка БЛ, остановка, запуск и т.д.

Сервер операций на агентах - список серверов, входящих в операцию на агентах.

Журнал операций на агентах - сообщения, поступающие от агентов в процессе выполнения операции.

Сервер конвертации - список серверов конвертации, с которыми Хоттабыч настроен работать.

Последний сервер конвертации - содержит одну запись с указанием последнего использованного сервера конвертации.

Загрузка на CDN - информация о настройках загрузки дистрибутива на update.sbis.ru.

Очередь обновлений - предназначена для блокирования и постановки в очередь операций, которые не могут выполняться одновременно (на данный момент это актуально только для запрета одновременного обновления cloud-ctrl и его продуктов). По завершению операции запись из таблицы RpcCallQueue удаляется. В таблице хранятся параметры необходимого для запуска операции grpc-вызова. Подробнее об этом решении см. в C++ комментариях в методу Продукт.Обновить.

Продукт - список продуктов или webapp, с которыми Хоттабыч настроен работать.

ИсторияОперацийПродукта - список операций в данный момент или прошлом выполняемых на данном Хоттабыче. В этой таблице хранится информация о ходе операции и параметры её запуска.

ИсторияОперацийСервера - служит для группировки записей из таблицы ИсторияОперацийЛогике и задания общих параметров.

ИсторияОперацийЛогике - список логических единиц на которые разбита операция (см. таблицу ИсторияОперацийПродукта). Например, обновление конкретной бизнес-логики, конвертация конкретной базы, обновление агента на сервере или чтение определённой информации из дистрибутива и передача облаку.

ЖурналСостоянияОпераций - логи выполнения операций.

Таблицы

Описание актуальных таблиц находится на сайте wi.sbis.ru.

Типовые выборки

1. Получение списка облаков для отображения в интерфейсе приложения.

Для отображения в интерфейсе списка облаков, необходима информация в таблице "ОблачноеПриложение".

Алгоритм выборки:

Выбираем все записи из таблицы "ОблачноеПриложение".

Дополняем каждую запись информацией из таблицы "Репозиторий" по ключу "ОблачноеПриложение.Репозиторий".

2. Получение информации о конкретном облаке.

Для отображения в интерфейсе необходима информация, находящаяся в таблице "ОблачноеПриложение". При запросе будет передан конкретный идентификатор группы.

Алгоритм выборки:

Получаем запись по ключу "@ОблачноеПриложение" из таблицы "ОблачноеПриложение".

Дополняем информацией из таблицы "Репозиторий" по ключу "ОблачноеПриложение.Репозиторий".

3. Обновление информации о конкретном облаке.

Сохранение записи облака в БД

Алгоритм:

Обновляем запись в таблице "ОблачноеПриложение", по ключу "ОблачноеПриложение.@ОблачноеПриложение".

Если указан пароль, то обновляем запись в таблице "CloudPassword" по ключу "ОблачноеПриложение.@ОблачноеПриложение".

4. Получение списка новостей.

Список новостей выводится постранично упорядоченный по дате публикации, с возможностью фильтрации.

Алгоритм выборки:

Из таблицы "UpdateNews" получаем список новостей.

Из таблицы "UpdateNewsProduct" получаем список связей, с помощью таблицы "Product" получаем поля "Name" и "Distribution" и группируем список по новости.

Заполняем поля для каждой новости из списка с продуктами.

5. Получение списка новостей по продукту.

Список новостей по приложению используется в мастере создания обновления для выбора подходящей новости.

Алгоритм выборки:

Из таблицы "UpdateNewsProduct" выбираются идентификаторы новостей, связанные с приложением по идентификатору "Product".

Из таблицы "UpdateNews" выбираются новости, идентификаторы которых были отобраны в предыдущем шаге.

С помощью таблицы "Product" получаем поля "Name" и "Distribution".

Заполняем поля для каждой новости.

6. Обновление информации о конкретной новости.

Сохранение записи новости в БД

Алгоритм:

Обновляем запись в БД, по ключу "UpdateNews.@UpdateNews".

7. Получение списка пользователей.

Список пользователей используется для управления доступом пользователей и групп пользователей к приложению.

Алгоритм выборки:

Отправляется запрос в облако для получения всего списка пользователей.

Из таблицы "ГруппаПользователь" выбираются все записи, попутно дополняя названиями групп из таблицы "ОписаниеГруппаПользователь".

Список пользователей, пришедший из облака, перебирается и дополняется данными из получившегося списка "ГруппаПользователь".

8. Получение информации о конкретном пользователе.

Для отображения в интерфейсе информации о пользователе.

Алгоритм выборки:

Запрашиваем облако информацию о пользователе, передав идентификатор пользователя.

Выбираем из таблицы "ГруппаПользователь" идентификаторы групп, которым принадлежит пользователь.

9. Обновление связей пользователя и групп.

Для добавления пользователя в группы в таблицу "ГруппаПользователь" добавляются записи, содержащие идентификатор пользователя и группы.

Алгоритм выборки:

Удаляем все записи по ключу "ГруппаПользователь.Пользователь"

Добавляем новые записи.

10. Получение списка групп пользователей.

Для объединения пользователей в группу в интерфейс выводится список групп.

Алгоритм выборки:

Выбирается весь список из таблицы "ОписаниеГруппаПользователь"

11. Обновление записи группы.

Сохранение записи группы пользователей в БД

Алгоритм:

Обновляем запись в БД, по ключу "ОписаниеГруппаПользователь.@ОписаниеГруппаПользователь".

12. Обновление связей приложения, пользователей и групп.

В настройках приложения указываются пользователи и группы пользователей, которые могут запускать операции над приложением.

Алгоритм:

Получаем список пользователей и групп по ключу "Продукт"

Сравниваем новый список и список полученный из выборки. Формируем списки на удаление и добавление.

Удаляем записи по ключам "Продукт" и "Пользователь" или "ОписаниеГруппаПользователь"

Добавляем новые записи.

15. Выборка клиентов скрипта принадлежащих определенной БД, с определенным статусом.

Требуется получать клиентов, на которых скрипт выполнен, были ошибки, или выполняется в данный момент.

Алгоритм выборки:

Делаем запрос к таблице "ClientDatabase" с фильтром по полям "ИсторияОперацийЛогика", "Status". Доступ по индексу ClientDatabase (ИсторияОперацийЛогика, Status)

16. Выборка клиентов скрипта, по которым не было ответа в течение некоторого времени.

Требуется получать клиентов, на которых скрипт завис, то есть, не было отчета о выполнении дольше таймаута.

Алгоритм выборки:

Делаем запрос к таблице "ClientDatabase" с фильтром по полям "Status" = 3, "Timestamp" < дата. Доступ по индексу ClientDatabase (Status=3). Записей со статусом 3 (выполняется) всегда небольшое количество, поэтому индекса достаточно для выборки.

17. Выборка баз и обновлений, с необработанными клиентами.

Требуется отобразить обновления и базы данных, в которых присутствуют клиенты, где скрипт завис, то есть, не было отчета о выполнении дольше таймаута.

Алгоритм выборки:

Делаем запрос к таблицам ИсторияОперацийПродукта, отбираем активные обновления по индексу ИсторияОперацийПродукта(Окончание=NULL), соединяемся по внешним ключам с таблицами ИсторияОперацийСервиса, ИсторияОперацийЛогике, фильтруем незавершенные логики баз данных. С помощью LATERAL подзапроса к таблице ClientDatabase отбираем последнее время обновления для каждой БД по индексу ClientDatabase (ИсторияОперацийЛогике, Timestamp). Таким образом получаем список баз, где обработка не завершена, но и ничего не обрабатывается.

18. Построение отчета выполнения скрипта по нескольким приложениям.

Один и тот же скрипт выполняют на нескольких приложениях. В интерфейсе требуется отображать сводный отчет по выполнению скрипта на всех приложениях.

Алгоритм выборки:

результат выполнения одного скрипта на одном приложении записывается в таблицу ScriptHistory.

Идентифицирующим скрипт полем является md5 от содержимого скрипта.

19. Выборка задач, которые доступны для запуска для заданного обновления.

Требуется получать идентификатор задач, которые необходимо выполнить для нормального продолжения процесса обновления.

Алгоритм выборки:

Делаем запрос к таблице "Task" с фильтром по полям "UpdateId" = @ИсторияОперацийПродукта, "Status" = 4 и проверкой отсутствия записей в таблице "Task" с фильтром "Status" != 0 для тех идентификаторов задач ("@Task") которые указаны в таблице "TaskRelationship" в столбце "Dependency" для записей "Target" которых равен отобранному ранее идентификатору задач. Доступ по индексу Task (UpdateId, Status) и индексу TaskRelationship(Target, Dependency)

20. Выборка задач по статусу и идентификатору обновления.

Требуется получать идентификатор задач, которые были прерваны.

Алгоритм выборки:

Делаем запрос к таблице "Task" с фильтром по полям "UpdateId" = @ИсторияОперацийПродукта, "Status" = 1. Доступ по индексу Task (UpdateId, Status).

21. Выборка выполняющихся операций.

Требуется отображать, текущий статус выполнения операций, прогресс или ошибки.

Алгоритм выборки:

Делаем запрос к таблице "ИсторияОперацийПродукта" с фильтром по полю "Окончание" = null. Доступ по индексу ИсторияОперацийПродукта (Окончание).

22. История выполненных операций.

Требуется получить отсортированный по времени начала запуска список завершённых операций.

Алгоритм выборки:

Делаем запрос к таблице "ИсторияОперацийПродукта" с фильтром по полю "Окончание" != null. Доступ по индексу ИсторияОперацийПродукта (Начало).

23. Выборка информации об операции.

Для выполнения любых действий операции требуется иметь доступ к информации об этой операции (её тип, логики и их параметры, идентификаторы в облаке и т.д.).

Алгоритм выборки:

Делаем запрос к таблице ИсторияОперацийПродукта по идентификатору, отбираем связанные записи в таблицах ИсторияОперацийСервиса, ИсторияОперацийЛогике и Продукт.

24. Выборка логов операции.

Требуется отобразить общие логи операции.

Алгоритм выборки:

Делается запрос к таблице ЖурналСостоянияОпераций с сортировкой по времени и отбором только записей связанных с заданной операцией по ключу из таблицы ИсторияОперацийПродукта.

25. Выборка структуры продуктов по разделам.

Требуется отобразить сгруппированный по директориям список продуктов конкретного облака с которыми можно выполнять операции.

Алгоритм выборки:

Делается запрос к таблице Продукт отбором только записей связанных с заданным облаком по ключу из таблицы ОблачноеПриложение, для получения продуктов из определённой группы используется ключ Раздел.

26. Выборка сценариев выкладки для приложения.

Требуется получить список сценариев для определенного приложения.

Алгоритм выборки:

Делаем запрос к таблице AppUpdateTemplate с отбором записей связанных с заданным приложением по ключу из таблицы Продукт.

27. Выборка версий со сценариями

Требуется получить список версий со сценариями, относящихся к приложению.

Алгоритм выборки:

28. Выборка сервера конвертации

Требуется получить информацию по определенному серверу конвертации.

Алгоритм выборки:

Делаем запрос к таблице ConverterServer с отбором записей по определенному Хосту.

29. Выборка истории загрузки конвертера.

Получение истории загрузки определенного сервера конвертации.

30. Выборка только активных задач конвертации.

Требуется отобрать только активные задачи для серверов конвертации.

Алгоритм выборки:

Делаем запрос к таблице ConverterServer, соединяем по внешнему ключу с таблицей ConverterTaskHistory с фильтром "EndTime" is null.

31. Выборка приложений из истории скрипта с определенным md5 скрипта.

Требуется отобрать скрипт из истории для определенного приложения с определенным md5 скрипта.

32. Поиск вложения по md5 файла.

Требуется отобрать вложения для обновления по md5 файла.

Алгоритм выборки:

Выполняем запрос к таблице ИсторияОперацийПродукта, соединяем по внешнему ключу с таблице UpdateAttach, фильтруем по полям @ИсторияОперацийПродукта и md5.

33. Поиск вложения по идентификатору файла.

Требуется отобрать вложения для обновления по идентификатору файла.

Алгоритм выборки:

Выполняем запрос к таблице ИсторияОперацийПродукта, соединяем по внешнему ключу с таблице UpdateAttach, фильтруем по полям @ИсторияОперацийПродукта и Ид.

34. Отбор всех пользователей, внесенных в группы.

Получение списка пользователей внесенных в группы.

Алгоритм выборки:

Делаем запрос к таблице ОписаниеГруппаПользователь с фильтром по полю "НазваниеГруппы" IS NOT NULL.

Индексы

Индекс	Выборки
Продукт (ОблачноеПриложение)	1, 2, 3
Продукт (ОблакоВнешнийИд)	1, 2, 3
UpdateNewsProduct (UpdateNews)	4, 5, 6
UpdateNewsProduct (Product)	4, 5, 6
ПродуктПользователь (Пользователь)	7, 8, 9
ГруппаПользователь (ОписаниеГруппаПользователь, Пользователь)	7, 8, 9, 10
Продукт (Продукт, Пользователь)	11, 12

ClientDatabase (ИсторияОперацийЛогике, Status)	15
ClientDatabase () WHERE "Status"=3	16
ClientDatabase (ИсторияОперацийЛогике)	17
ScriptHistory (md5, ProductID)	18
Task (UpdateId) WHERE "Status" = 4	19, 20
TaskRelationship (Target)	19
TaskRelationship (Dependency)	19
ИсторияОперацийПродукта (Окончание)	21
ИсторияОперацийПродукта (Продукт, Начало)	22
ИсторияОперацийПродукта (Продукт)	23
<pre> 1 case 2 when(3 "Окончание" is not null 4) then 5 null 6 else 7 1 8 end 9 case 10 when(11 "ТипОбновления" = 6 OR 12 "ТипОбновления" = 13 13) then 14 null 15 else 16 1 17 end </pre>	
ИсторияОперацийСервиса (ИсторияОперацийПродукта)	23
ИсторияОперацийЛогике (ИсторияОперацийСервиса)	23
ЖурналСостоянияОпераций (ИсторияОперацийПродукта, ДатаВремя)	24
ЖурналСостоянияОпераций (ИсторияОперацийПродукта) WHERE "ИсторияОперацийЛогике" is null	24
ЖурналСостоянияОпераций (ИсторияОперацийЛогике)	24
Продукт (ОблачноеПриложение)	25
Продукт (ОблачноеПриложение, ВнешнийИд)	25
AppUpdateTemplate(ProductId)	26
AppVersion(ProductId)	27
ConverterServer(Host)	28
ConverterTasksHistory(ConverterServer)	29
ConverterTasksHistory("EndTime" is null)	30
ScriptHistory(md5, ProductID)	31
UpdateAttach(md5)	32
UpdateAttach(Ид)	33

ОписаниеГруппаПользователь(НазваниеГруппы)	34
--	----

Публичное API

Функционал не содержит публичного API.

Организация кода

Модули, их зависимости

Вся реализация находится в модуле "Version manager" Хоттабыча.

Сторонние библиотеки

Не используются.

Репозитории GIT

<https://git.sbis.ru/orchestration/admin.git>

Диаграмма классов

Не актуально.

Подсистема распределения прав

Роли

Администратор Хоттабыча - позволяет создавать, изменять и удалять дистрибутивы и сборки в хранилище дистрибутивов, а также строить отчеты анализа расхождения БД с дистрибутивом.

Оператор хранилища дистрибутивов - позволяет создавать, изменять и удалять дистрибутивы и сборки в хранилище дистрибутивов, а также строить отчеты анализа расхождения БД с дистрибутивом.

Приложение Хоттабыч в любом облаке кроме prod - СОТРУДНИКАМ ТЕНЗОРА НЕ УСТАНОВЛИВАЕТСЯ!

Предназначена для установки на приложение Хоттабыч, через аутентификацию которого другие сервисы системы обновлений получают доступ к сервисам облака. Например, планы выпуска обращаются в облако для регистрации параметров или в датацентр для операций над контейнерами.

Приложение Хоттабыч в облаке prod - СОТРУДНИКАМ ТЕНЗОРА НЕ УСТАНОВЛИВАЕТСЯ! Предназначена для установки на приложение Хоттабыч, через аутентификацию которого тестовые стенды получают доступ к рабочему сервису планов выпуска. Ограничивает редактирование планов с тестовых стендов.

Участки системы

Обновления - дооступ к управлению обновлениями в Хоттабыче.

Настройки Хоттабыча - доступ к разделу Настройки аккордеона Хоттабыча.

Настройки приложений - изменение настроек приложений в Хоттабыче в разделе Приложения.

Анализ БД - построение отчетов анализа изменения БД.

Параметры облака, задачи планировщика

Параметры облака

Название	Описание	Значение по умолчанию	Тип
Управление	Интервал между	60	Число

версиями.Агенты.Keepalive	сообщениями Фаза в процесса выполнения от агента (определяет keepalive агента, если агент не ответит в период, равный интервалу - логика завершится с ошибкой).		
Управление версиями.Агенты.АктуальнаяВерсия	Версия агента, если ее необходимо подменить.	—	Число
Управление версиями.Агенты.ЛимитСвободногоМеста.ДляЗагрузкиУтилиты	Минимальный объём свободного места для загрузки утилиты.	300	Число
Управление версиями.Агенты.ЛимитСвободногоМеста.НаВсехФазах	Минимально допустимый объём свободного места для всех фаз, кроме доставки. Проверяется агентом на сервере логики.	100	Число
Управление версиями.Агенты.ЛимитСвободногоМеста.НаФазеДоставки	Минимально допустимый объём свободного места на фазе доставки. Проверяется агентом на сервере логики.	1000	Число
Управление версиями.Агенты.Порт	Параметр определен в Хоттабыче. Здесь используется для отправки команд на перезапуск / остановку узлов.Определяет порт обслуживания вызовов методов на Агенте Хоттабыча	2013	Число
Управление версиями.Анализ БД.Период хранения	Срок в месяцах, через который удаляются отчеты анализа БД после создания.	24	Число
Управление версиями.Временное.ИсползоватьОповещениеЗависимыхСервисовВместоПерезапуска	Вместо перезапуска зависимых сервисов использовать старый вариант оповещения через вызов метода на узлах.	Нет	Логический
Управление версиями.Временное.Копировать дистрибутивы через хранилище	Если Да, то включает копирование дистрибутивов на стороне хранилища. Иначе копирование происходит в хоттабыче.	Нет	Логический
Управление версиями.Временное.НеоблачныеПриложенияСНестандартнымФорматомВерсии	Список необлачных приложений, для которых необходимо выбрасывать номер патча из версии.	[Sbis3Plugin]	Массив строк
Управление версиями.Временное.Скрипты.ОтправлятьСтатистику	Если Нет то в ini скриптового узла параметр Ядро.Статистика облака.ОтправлятьСтатистику устанавливается в Нет. Иначе упомянутый параметр не изменяется.	Да	Логический
Управление версиями.Конвертеры.Врем	Максимальное время ожидания ресурсов перед	30	Число

яОжиданияРесурсов	запуском задачи.		
Управление версиями.Конвертеры.ВремяХраненияХешаСхем	Время хранения хеша схем для обновления типа Подготовка для БД.	2	Число
Управление версиями.Конвертеры.ЛимитЗадач	Если суммарный вес задач на конвертере превышает указанное значение новые задачи не запускаются.	70	Число
Управление версиями.Конвертеры.ЛимитПроцессора	Если загрузка процессора на конвертере превышает указанное значение новые задачи не запускаются.	95	Число
Управление версиями.Конвертеры.ЛимитПроцессораХоста	Если загрузка процессора на хосте конвертера превышает указанное значение новые задачи не запускаются.	95	Число
Управление версиями.Конвертеры.ПиковоеПотреблениеПамятиДжинном	Максимально возможный объём потребляемой памяти одним процессом jinnеe.	2400	Число
Управление версиями.Обновления.MultithreadedHashingНаПодготовке	Задаёт значение multithreaded-hashing в настройках алгоритма конвертации баз на фазе подготовки.	Да	Логический
Управление версиями.Обновления.UrlCDN	Url для запроса серверов CDN.	/cdn/	Строка
Управление версиями.Обновления.UrlСервисаСтатики	Url для запроса серверов статики.	/static/	Строка
Управление версиями.Обновления.Время хранения дистрибутива	Сколько хранить данные дистрибутивов в базе сервиса с момента последнего использования.	7	Число
Управление версиями.Обновления.ВремяХраненияИстории	Число месяцев хранения истории обновлений приложений Хоттабыча. Не входящая в интервал история будет удалена.Минимальное значение - 1Максимальное значение - 11	3	Число
Управление версиями.Обновления.ЗапретитьОбновлятьНаУстаревшиеВерсии	Запрещает обновлять приложение на версию больше текущей, но с датой меньше текущей.	Нет	Логический
Управление версиями.Обновления.ЗапретОбновленийВоВремяКонвертацииОблака	При запуске обновлений проверять, не запущено ли конвертация облака и наоборот	Да	Логический
Управление версиями.Обновления.Класстеры.Подставлять рекомендованную версию	Подставлять в мастере запуска обновления кластера рекомендованную для обновления версию. Если параметр в значении Нет,	Да	Логический

	то версию нужно будет выбрать вручную.		
Управление версиями.Обновления.КлиентПродуктовогоОблака.ИД	Идентификатор клиента продуктового облака	334001030726220	Строка
Управление версиями.Обновления.КлиентПродуктовогоОблака.Секрет	Секрет продуктового облака	—	Строка
Управление версиями.Обновления.КлиентПродуктовогоОблака.СекретныйКлюч	Секретный ключ продуктового облака	—	Строка
Управление версиями.Обновления.ЛегкаяКонвертация.ПроверятьДублирующиеСервера	Параметр позволяет отключить проверку дублирующих серверов при легкой конвертации. По умолчанию проверка включена.	Да	Логический
Управление версиями.Обновления.Легкое.ПоследовательныйРазворотДанных	Производить ли при лёгком обновлении развёртывание данных последовательно внутри сервиса?	Да	Логический
Управление версиями.Обновления.Легкое.РазворотДанных	Параметр включающий разворот данных при легком обновлении.	Нет	Логический
Управление версиями.Обновления.ЛимитОшибокПрогрева	Количество возвращаемых ошибок от сервера во время прогрева сервиса.	6	Число
Управление версиями.Обновления.Логи.МинимальныйПериодХранения	Минимальный период хранения логов завершённых обновлений	14	Число
Управление версиями.Обновления.Логи.ПодробноеЛогированиеУтилиты	Если данный параметр установлен в значение true, то на обновляемом сервере будет производиться более подробное логирование процесса обновления.	Нет	Логический
Управление версиями.Обновления.Логи.ПодробныеСообщенияДжинна	Должен ли агент транслировать в журнал сообщений подробные сообщения джина о ходе конвертации	Да	Логический
Управление версиями.Обновления.Мастер-приложения с общей БД	Список мастер-приложений, которые используют общую БД. При запуске обновления с конвертацией такого приложения, будет происходить запуск обновлений всех связанных.	—	Массив строк
Управление версиями.Обновления.Мастер-реплика.ДополнительныйТаймаутНормализацииЛага	Время ожидания нормализации лага реплики после конвертации мастера складывается из двух	300	Число

	составляющих:1) времени прохождения основной фазы мастер-реплики2) времени, указанном в этом параметре.		
Управление версиями.Обновления.МинимальноНеобходимоеСвободноеМестоНаРегистраторе	Минимально необходимый объем свободного места в директории ApplicationDataPath для скачивания/распаковки дистрибутива metadata-registrator'ом.	2048	Число
Управление версиями.Обновления.Необлачное.ОжиданиеСинхронизацииБалансировщика	Таймаута ожидания синхронизации балансировщика, при облачном обновлении.	86400	Число
Управление версиями.Обновления.НеПроверятьMd5Сервиса	Список сервисов для которых при обновлении по частям сервисов не будет проверяться md5 по завершении основной фазы. (Используется регулярное выражение, т.е. если указать * то проверка будет отменена для всех сервисов, *-ps - не будут проверяться сервисы представления.)	—	Массив строк
Управление версиями.Обновления.НеУстанавливатьReadOnlyДляСервиса	Для сервисов из этого списка при обновлении не будет установлено readonly на каталог сервиса на обновляемой БЛ. (Используется регулярное выражение, т.е. если указать * то не устанавливаем readonly для всех сервисов, *-ps - для всех сервисов представления.)	—	Массив строк
Управление версиями.Обновления.ОжиданиеЗапускаБЛ	Срок ожидания запуска узла сервиса. По истечении срока узел сервиса пометится ошибкой.	300	Число
Управление версиями.Обновления.ОжиданиеОтчётаХоттабыча	Время в течение которого ожидается отчёт о ходе обновления от БЛ Хоттабыча.	—	Число
Управление версиями.Обновления.ОжиданиеПослеУстановкиРежимаBoth	Если в обновлении участвуют сервисы представления, то после установки режима обновления both необходимо время на подгрузку ресурсов, прежде чем направлять пользователя на обновлённую логику.	30	Число
Управление версиями.Обновления.ОстанавливатьсяПослеОсновнойФазы	Для приложений из этого списка, если не установлен флаг Останавливаться только на ошибках после	—	Массив строк

	основной фазы будет совершена остановка, и фазу подтверждения необходимо будет запустить вручную.		
Управление версиями.Обновления.ОтключитьРегистрациюДляПриложений	Список приложений, для которых не нужно выполнять регистрацию метаданных при обновлении.	—	Массив строк
Управление версиями.Обновления.Полное.ЧислоЗадержекРазблокировкиПользователей	Коэффициент, который отображает сколько временных интервалов равных значению параметра Управление облаком.СтраницаБлокировки.ЗадержкаРазблокировкиПользователей необходимо ожидать перед тем как выдать ошибку, в случае, если % разблокированных пользователей не меняется.	20	Число
Управление версиями.Обновления.Последовательное.ПодготовкаПерсональныхПриложений	В случае, если задан данный параметр, то для персональных сервисов пройдет последовательный вариант проведения подготовки БЛ внутри tenant-группы.	Нет	Логический
Управление версиями.Обновления.Последовательное.РазбросУведомленийИнтерфейса	При наличии сервисов представления при последовательном обновлении данный параметр устанавливает диапазон в котором в случайное время будет показываться уведомление об обновлении. Это обеспечит разброс запросов от клиентов в переходе на новые узлы сервиса представления.	1200	Число
Управление версиями.Обновления.Посхемное.ИнтервалУведомлений	Интервал между уведомлением клиента и началом обновления. Он же интервал между уведомлением следующей группы клиентов.	60	Число
Управление версиями.Обновления.Посхемное.КоличествоУведомляемыхКлиентов	Количество клиентов на хост БД, уведомляемых в рамках одного интервала уведомлений.	2000	Число
Управление версиями.Обновления.Посхемное.ТаймаутОжиданияМетодовКлиента	Время ожидания конвертером завершения исполнения методов под сессией клиента на сервисе БЛ после блокировки клиента. После окончания таймаута конвертация запустится безусловно.	600	Число

Управление версиями.Обновления.Посхемное.УведомлятьКлиентов	Нужно ли отсылать клиентам уведомление об обновлении с возможностью его отложить.	Да	Логический
Управление версиями.Обновления.Посхемное.ЧислоПотоковКонвертераНаХост	Количество потоков конвертера на хост при посхемной конвертации	6	Число
Управление версиями.Обновления.ПредупреждатьЕслиНеЗаданаРеплика	Показывать или нет сообщение У одной из баз приложения не задана реплика. Вы уверены, что остановили все слейвы?	Да	Логический
Управление версиями.Обновления.Приложения обновляемые по предыдущему стенду	Имена приложений, при запуске обновлений которых на версию без номера сборки, номер сборки определяется по предыдущему стенду.	[cloud-ctrl]	Массив строк
Управление версиями.Обновления.Приложения с общей БД	Список приложений, которые используют общую БД. При запуске обновления с конвертацией одного приложения, будет происходить запуск обновлений всех связанных.	—	Массив строк
Управление версиями.Обновления.Пропускать запуск приложений с общей БД	Список приложений, которые не должны участвовать в схеме обновления приложений с общей БД.	—	Массив строк
Управление версиями.Обновления.ПропускатьЗапускСлейвов	Пропускать запуск обновлений slave-приложений.	Нет	Логический
Управление версиями.Обновления.Разворот данных.Таймаут запуска	Время за которое сервис должен ответить, что разворот запущен, иначе обновление упадёт.	5	Число
Управление версиями.Обновления.РегистрацииВрежимеОтладки	Список регистраций, для которых будут игнорироваться ошибки.В параметре задаются:а) для обычных регистраций - название объекта-регистратора из *.ogx. Всегда имеют префикс DistrReg_*, например, DistrReg_Settingsб) для упрощенных регистраций - по формату SimplifiedRegistration.<расширение_регируемых_файлов>.Например, SimplifiedRegistration.link	—	Массив строк
Управление версиями.Обновления.РегистрацияРесурсов.РегистрироватьНаПродеИз	Список контуров, при выполнении метарегистрации из которых ресурсы будут	[]	Массив строк

	регистрироваться в продуктовом облаке		
Управление версиями.Обновления.ТаймаутВызоваКритичныхРегистраций	Таймаут вызова критичных регистраций.	600	Число
Управление версиями.Обновления.ТаймаутВызоваОтложенныхРегистраций	Таймаут вызова отложенных регистраций.	300	Число
Управление версиями.Обновления.ТаймаутЗапускаБЛ	Срок ожидания запуска бизнес-логики. По истечении срока бизнес-логика пометится ошибкой. Имеет формат json-объекта, где ключами являются названия приложений. Ключ all задаёт таймаут для всех остальных случаев.	{all:300}	Объект
Управление версиями.Обновления.ТаймаутОжиданияНачалаЗагрузкиСтатики	Таймаут ожидания начала загрузки статики после отправки команды. Нет смысла задавать меньше чем ОжиданиеОтчётаХоттабыча.	15	Число
Управление версиями.Обновления.ТаймаутОжиданияОтветаАгента	Таймаут ожидания отчётов агента при опросе местоположения логик.	10000	Число
Управление версиями.Обновления.ТаймаутПерезапускаЗависимыхСервисов	Время ожидания перезапуска зависимых сервисов в минутах. Если 0 или не задан - ожидать бесконечно.	—	Число
Управление версиями.Обновления.ТаймаутПрерывания	Таймаут ожидания операции прерывания.	120	Число
Управление версиями.Обновления.ЧерныйСписокРегистраторов	Список регистраций для исключения из обновления.В параметре задаются:а) для обычных регистраций - название объекта-регистратора из *.org. Всегда имеют префикс DistrReg_*, например, DistrReg_Settingsб) для упрощенных регистраций - по формату SimplifiedRegistration.<расширение_регистрируемых_файлов>.Например, SimplifiedRegistration.link	[]	Массив строк
Управление версиями.Обновления.ЧислоПотоковОбновленияАгентов	Число потоков обновления агентов при запуске операции из Управления облаком.	10	Число
Управление версиями.Общие.ВременнаяДиректория	Путь до директории хранения временных	—	Строка

	данных сервиса Version manager		
Управление версиями.Общие.ИмяСервиса.file-transfer	Используется в endpoint для запросов к FileTransfer	file-transfer	Строка
Управление версиями.Общие.Название продукта.РегистрацияМета данных	Название продукта Регистрация метаданных.	metadata-registrator	Строка
Управление версиями.Общие.Название продукта.УправлениеОблаком	Возможные названия дистрибутива продукта Управление Облаком.	[cloud-ctrl]	Массив строк
Управление версиями.Общие.Название продукта.ХранилищеФайлов	Возможные названия дистрибутива продукта Хранилище файлов.	[filestorage]	Массив строк
Управление версиями.Общие.ПороговоеВремяСинхронногоВызова	Для внешних вызовов длительностью выше порога будут писаться предупреждения в логи.	30	Число
Управление версиями.Общие.ТаймаутВызововНаДругиеСтенды	Таймаут вызовов на тестовые стенды	5000	Число
Управление версиями.Общие.ТаймаутЗапускаРаботыНаДругихСтендах	Таймаут запуска работ на других стендах.	60000	Число
Управление версиями.Скрипты.ИспользоватьПотокиВместоПроцессов	Использовать потоки вместо процессов для скриптовых БЛ.	Да	Логический
Управление версиями.Скрипты.КоличествоОшибочныхКлиентовПередПрерыванием	Количество клиентов с ошибками выполнения скрипта для его автоматического прерывания.	—	Число
Управление версиями.Скрипты.МаксимальноеЧислоЛогов	Максимальное число файлов логов, которые могут создаваться на одной бизнес-логике	20	Число
Управление версиями.Скрипты.МаксимальноеЧислоПроцессовСкриптовыхБЛ	Максимальное число процессов скриптовых БЛ.	10	Число
Управление версиями.Скрипты.МаксимальныйРазмерДанных	Ограничение размера данных одного клиента в МБ при записи в Redis хранилище скриптов	5	Число
Управление версиями.Скрипты.МаксимальныйРазмерЛога	Ограничение на размер файла лога, создаваемого в скрипте	100	Число
Управление версиями.Скрипты.Ограничение нагрузки обработки отчетов	Процент воркеров, занятых обработкой отчетов выполнения скрипта.Используется для ограничения нагрузки на	25	Число

	хоттабыч во время выполнения скриптов.		
Управление версиями.Скрипты.ОжиданиеОтчета	Количество минут ожидания отчета выполняющегося скрипта, спустя которое, скрипт будет помечен ошибкой.	5	Число
Управление версиями.Скрипты.Предел использования диска для перепакровки	Максимальный процент использования диска во время перепакровки логов скрипта. При превышении процента, операция будет прервана.	70	Число
Управление версиями.Скрипты.РазмерЗадачи	Число клиентов, отправляемых в задачу DWC выполнения скриптов	20	Число
Управление версиями.Скрипты.РазмерПачкиСообщений	Количество сообщений извлекаемых за раз в задаче по обработке результатов выполнения скриптов.	10000	Число
Управление версиями.Скрипты.РежимИсполненияСкриптов	Параметр, сигнализирующий о нахождении сервиса в режиме исполнения скриптов.	Нет	Логический
Управление версиями.Скрипты.Ссылка на ВНР	Ссылка на ВНР выполняющегося скрипта.	—	Строка
Управление версиями.Скрипты.Таймаут выполнения	Максимальное время в минутах, которое может выполняться задача выполнения скриптов на DWC	43200	Число
Управление версиями.Скрипты.Таймаут Задачи	Максимальное время в минутах, которое может выполняться задача выполнения скриптов на DWC	4320	Число
Управление версиями.Скрипты.Частота Отчетов	Частота с которой выполняющийся скрипт отчитывается о своем состоянии. Например, каждые 60 секунд.	60	Число
Управление версиями.Тестирование.АдресКонвертера	Предназначен для использования на локальных стендах. Для исключения взаимного влияния локальных стендов и основного Хоттабыча на конвертерах pre-test.	—	Строка
Управление версиями.Тестирование.ВерсияУтилиты	Для тестирования	—	Строка
Управление версиями.Тестирование.ВызыватьСкриптПодПользователем	Для тестирования	—	Логический

Управление версиями.Тестирование.ИспользоватьТестовыеПриложения	Включает функционал работы с тестовыми приложениями.	Нет	Логический
Управление версиями.Тестирование.Локальный планировщик задач	Параметр для включения замены функционала планировщика задач. Только для разработки на локальном стенде.	Нет	Логический
Управление версиями.Тестирование.ОстанавливатьсяНаРежимахОбновлений	При выполнении метода CloudLocker::SetUpdateMode в самом конце метода при установленном параметре и совпадении с именем обновляемого приложения выбрасывается исключение, что мы остановились по этому параметру. Это позволит после установки нужного режима тестированию проверить нужный функционал, а потом возобновить обновление.	—	Строка
Управление версиями.Тестирование.ОстанавливатьсяПослеЗахватаБлокировки	Для тестирования! Останавливать на заданное время работу утилиты после захвата блокировки, если для продукта задан параметр ОстанавливатьсяНаРежимахОбновлений.	—	Число
Управление версиями.Тестирование.ОстанавливатьсяПослеЗахватаБлокировкиВОбщемХранилище	Для тестирования! Останавливать на заданное время работу утилиты после захвата блокировки в общем хранилище перед загрузкой или удалением jinnее, если для продукта задан параметр ОстанавливатьсяНаРежимахОбновлений.	0	Число
Управление версиями.Тестирование.ОстанавливатьсяПриНедоступностиСкриптовогоУзла	Останавливаться на фазе очистки при неполучении информации об активности скриптового узла (поды) по stat.	Нет	Логический
Управление версиями.Тестирование.ПофазноеРазвёртывание	Имя продукта, для которого деплой будет производиться пофазно (без флага Непрерывное обновление).	—	Строка
Управление версиями.Тестирование.ПроверкаАсинхронныхЗапросов	Массив названий приложений, обновление которых должны останавливаться в следующих случаях:- перед каждой установкой/снятием режима обновления;- для режимов обновлений (mode: 6, 7) производить остановку	—	Массив строк

	перед началом конвертации клиентов (LockForUpdate) и перед разблокировкой (UnlockForUpdate).		
Управление версиями.Тестирование.РежимТестированияПроизводительности	Данный параметр выставляется true, для возможности запуска методов для тестирования, к примеру, LoadTest.PreparationProductForLoadTest или LoadTest.ResetProductAfterLoadTest.	Нет	Логический
Управление версиями.Тестирование.Сессия	Сессия, с которой запускается локальный планировщик	—	Строка
Управление версиями.Тестирование.СистемноеИмяСервисаВыпускаВерсий	Для тестирования	—	Строка

Задачи планировщика

Запуск задач вычисления прогресса применения правил - запускается каждый час, производит вычисление прогресса фазы применения правил для обновлений необлачных приложений.

Запуск задач обновлений - запускается каждые 20 секунд, запускает доступные для выполнения задачи обновлений.

Контроль разворота данных - запускается каждую минуту, проверяет ход выполнения обновлений типа разворот пакета данных.

Обновление статусов активных скриптов - запускается каждую минуту, следит за активными скриптами, запускает фазы выполнения скриптов на логике, либо завершает выполнение.

Обработка результатов выполнения скриптов - запускается каждую минуту, обрабатывает результаты выполнения скриптов в Хоттабыче и выводит их в журнал обновления.

Очистка данных скриптов - запускается раз в день в 12 часов, производит чистку данных завершенных скриптов. Учет выполнения скриптов генерирует большое количество записей в таблице ClientDatabase и зависящих от неё. Очистка делается через truncate таблицы, в то время когда скрипты не выполняются, или через удаление записей и VACUUM FULL используемых таблиц.

Очистка истории обновлений продуктов - запускается раз в день в 14 часов, удаляет устаревшую историю обновлений продуктов. Главная таблица таблица обновлений – ИсторияОперацийПродукта, записи из неё не удаляются, а только помечаются удаленными, чтобы иметь полную историю обновлений, удаляются лишь связанные сущности. Очистка таблиц связанных с ИсторияОперацийПродукта делается с помощью обычного удаления записей по внешнему ключу и каскадным удалением связанных записей. Факт очистки записывается в сервис истории.